Seguridad, Rendimiento y Disponibilidad de Servicios e Infraestructuras

Ana I. González Acuña

Grado en Ingeniería Informática



2026/27

SRDSI

1. Introducción a la seguridad informática
2. Seguridad en la comunicación
3. Seguridad en los servicios
4. Seguridad Operacional



Tema 2: Seguridad en la comunicación



2 Seguridad en la comunicación
2.1 Protocolos seguros de comunicación
2.2 Nivel de Aplicación
2.3 Nivel de Transporte
2.4 Nivel de Interred
2.5 Nivel de Enlace



2 Seguridad en la comunicación
2.1 Protocolos seguros de comunicación
2.2 Nivel de Aplicación
2.3 Nivel de Transporte
- SSL
- TLS
- DTLS
- QUIC
- Otros protocolos

2.4 Nivel de Interred
2.5 Nivel de Enlace

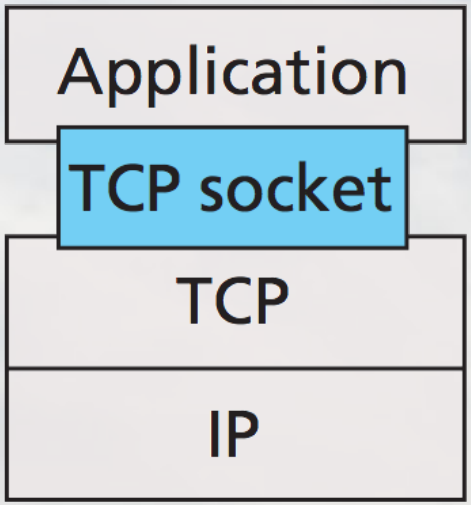


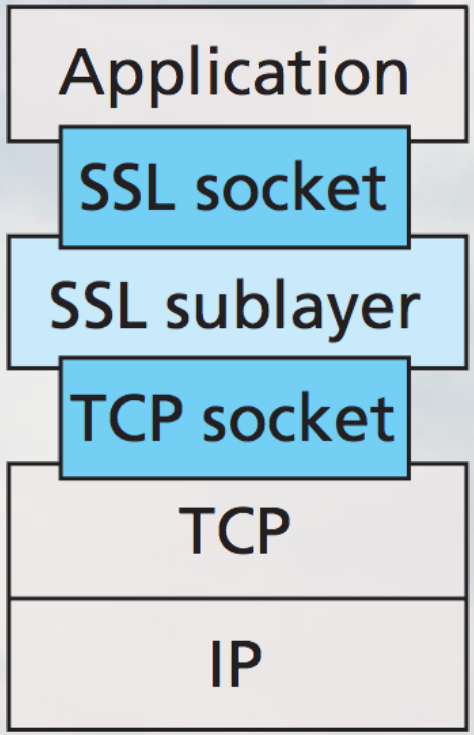
Secure **S**ocket **L**ayer (RFC 6101-*Historic*)

- Proporcionar servicios de seguridad extremo a extremo para aplicaciones en Internet mendiante una API-SSL, independientemente del protocolo de aplicación.
- Diseñado e implementado por Netscape (94). SSLv3 -> **OBSOLETO**
  Resumen de los eventos más importantes relacionados con SSL/TLS
- **Base** de Transport Layer Security (TLS)
- Requiere una PKI jerárquica (CAs y certificados X.509)
- Usos:
  - Objetivo inicial: proteger accesos web (https) autenticando al servidor y cifrando datos.
  - Otros protocolos clásicos: ftp, pop3, imap, smtp
  - Proxy SSL/TLS de cliente (stunnel), de red (openVPN)
- OpenSSL es un kit de herramientas criptográficas que implementan [SSL,] TLS y DTLS, junto con los estándares criptográficos que requieren. Otras opciones: GnuTLS, wolfSSL, MatrixSSL, ...



Dentro de TCP/IP





fuente: www-net.cs.umass.edu/cmpsci_591_453



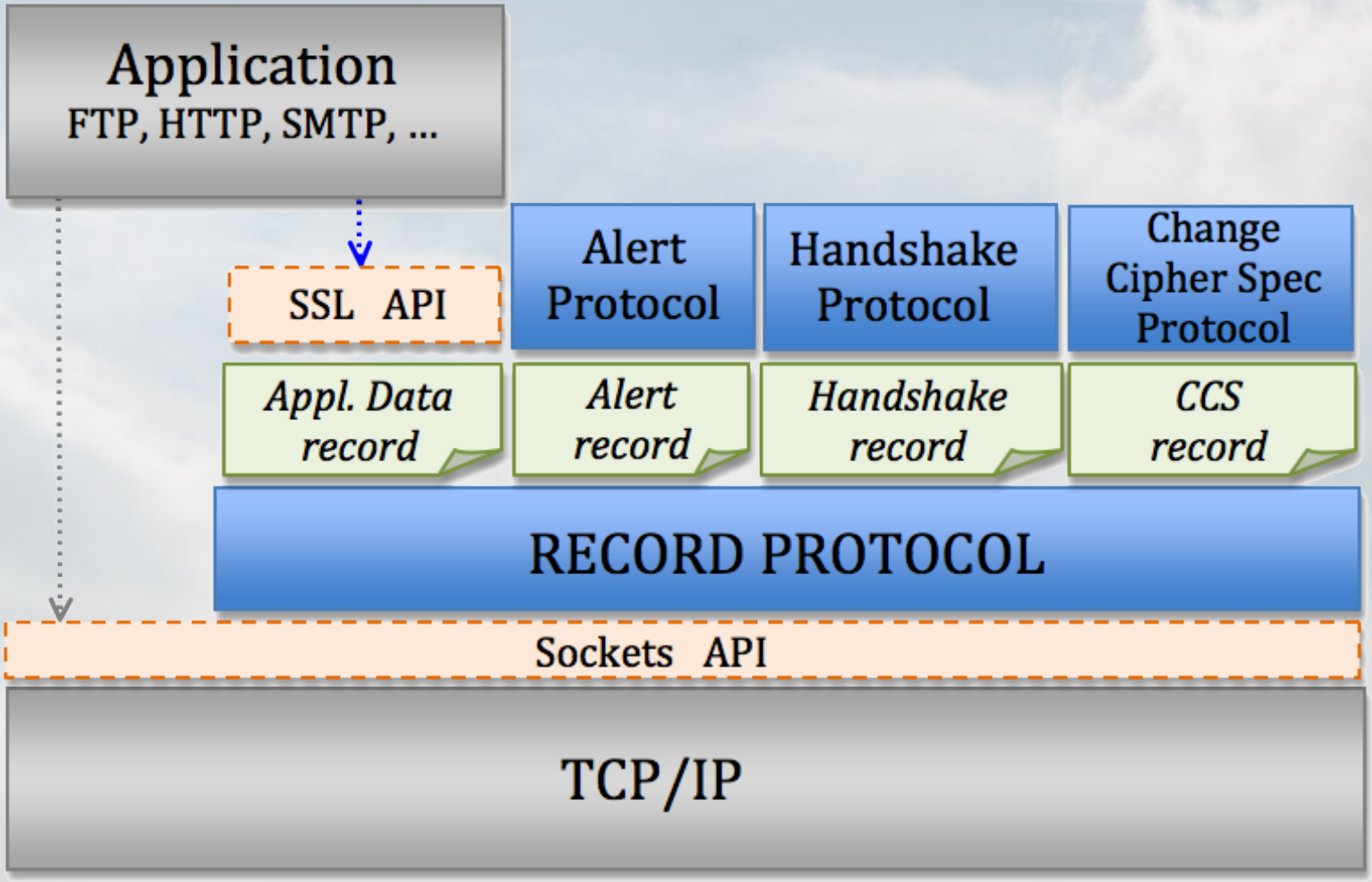
Servicios

- Confidencialidad de los datos
- Autenticación del servidor [y del cliente]
- Integridad de los datos

- Fragmentación
- Compresión (*deprecated*)
- Entramado



Arquitectura SSL(/TLS)



Protocolos SSL

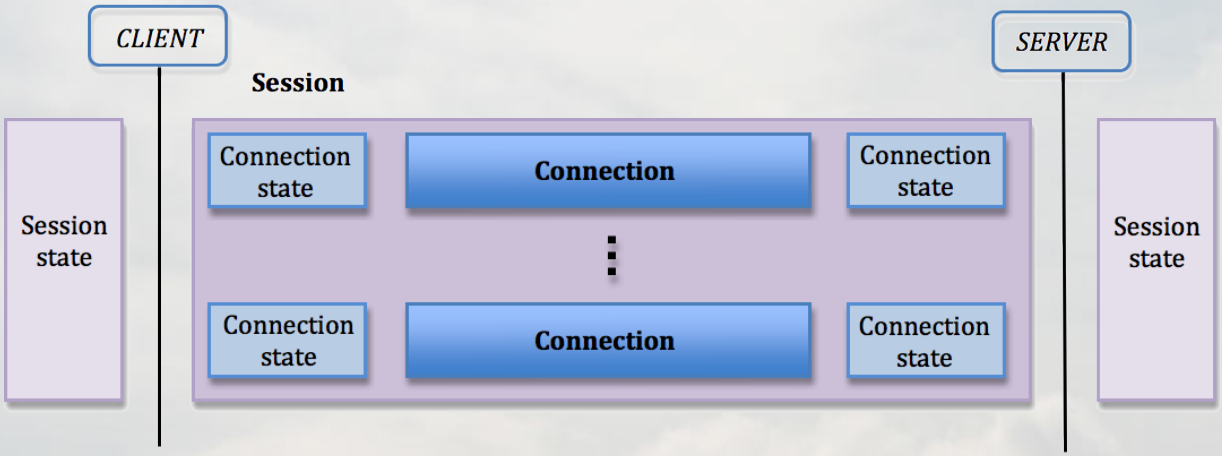
- **Record Protocol**, se utiliza como transporte seguro de los protocolos de las capas más altas (Confidencialidad, Integridad y/o compresión).
- **Handshake Protocol**:
  - Selección de los parámetros de sesión y conexión.
  - Autenticación del servidor [y cliente].
  - Establecimiento de sesión segura.
- **Change Cypher Spec. Protocol**, indicar al par que conmute a un nuevo conjunto de claves
- **Alert Protocol**, para avisar al par de cambios de estado (causa de fallos)



Sesión vs Conexión

Sesión: una asociación entre un cliente y un servidor (compartida por múltiples conexiones):
Session state: **ID sesión**, método de compresión, *cipher suite*, **master-secret**, certificado del par, *flag* para permitir nuevas conexiones

Conexión: conexión a nivel de transporte para **intercambiar datos** entre dos pares, asociada a una sesión:
Connection state: Números aleatorios compartidos, clave MAC, claves de sesión y IV para cliente y servidor, números de secuencia de mensajes enviados y recibidos.



Cipher suites

Conjunto de algoritmos que ayudan a segurizar una conexión en red. Extensible, esquema abierto a distintos algoritmos. Al establecer la sesión SSL/TLS, se negocia la combinación de algoritmos para:

- Intercambio y autenticación de claves (establecer una *pre-master secret*)
- Cifrado/descifrado
- Resumen (Hash)

`TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384`

Protocol TLS	Key Exchange	Key Authentication	Encryption	Bits	Cipher Mode	Message Authentication

`TLS_RSA_PSK_WITH_AES_128_GCM_SHA256`

(Distinto en TLSv1.3)

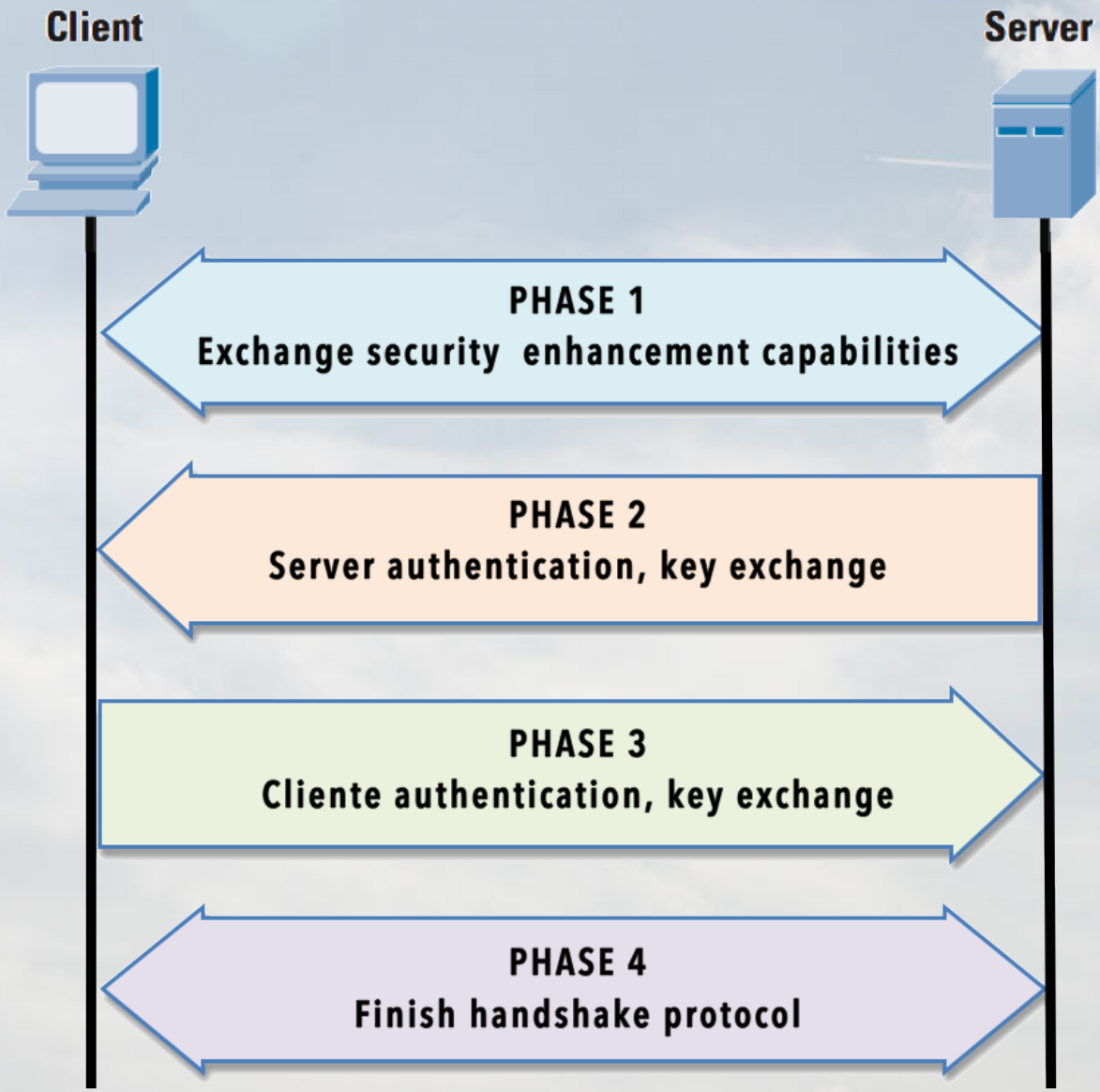
IANA's Cipher Suites 

SSL Handshake Protocol

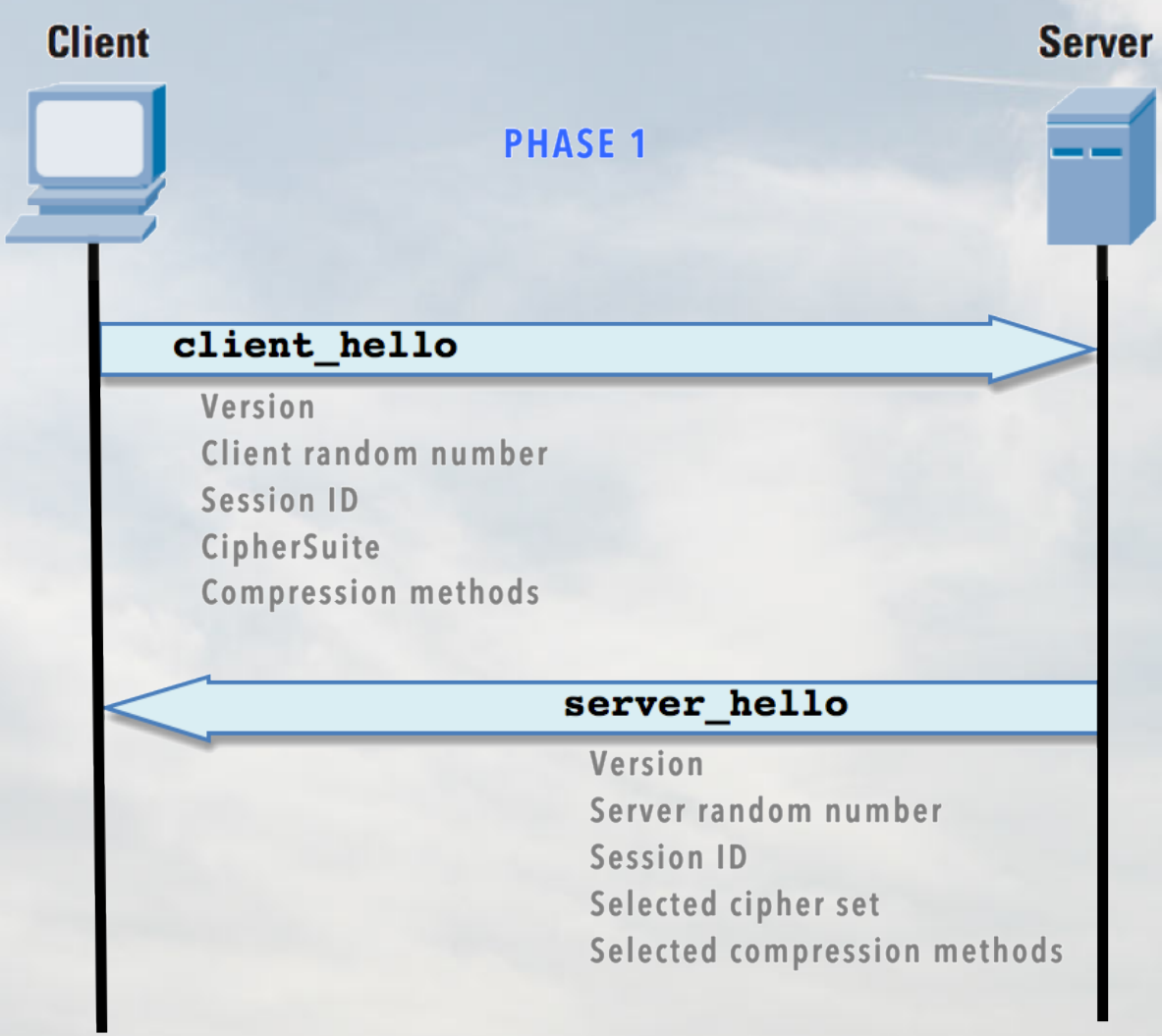
- Durante el protocolo *SSL Handshake*, el cliente y el servidor intercambian una serie de mensajes para negociar las mejoras de seguridad.
- Consta de cuatro fases:
  - Establecimiento de los parámetros de la comunicación.
  - Autenticación del servidor e intercambio de clave.
  - Autenticación del cliente e intercambio de clave.
  - Fase final.
    Tras esta fase ya se puede comenzar una sesión segura.

Explaining-ssl-handshake

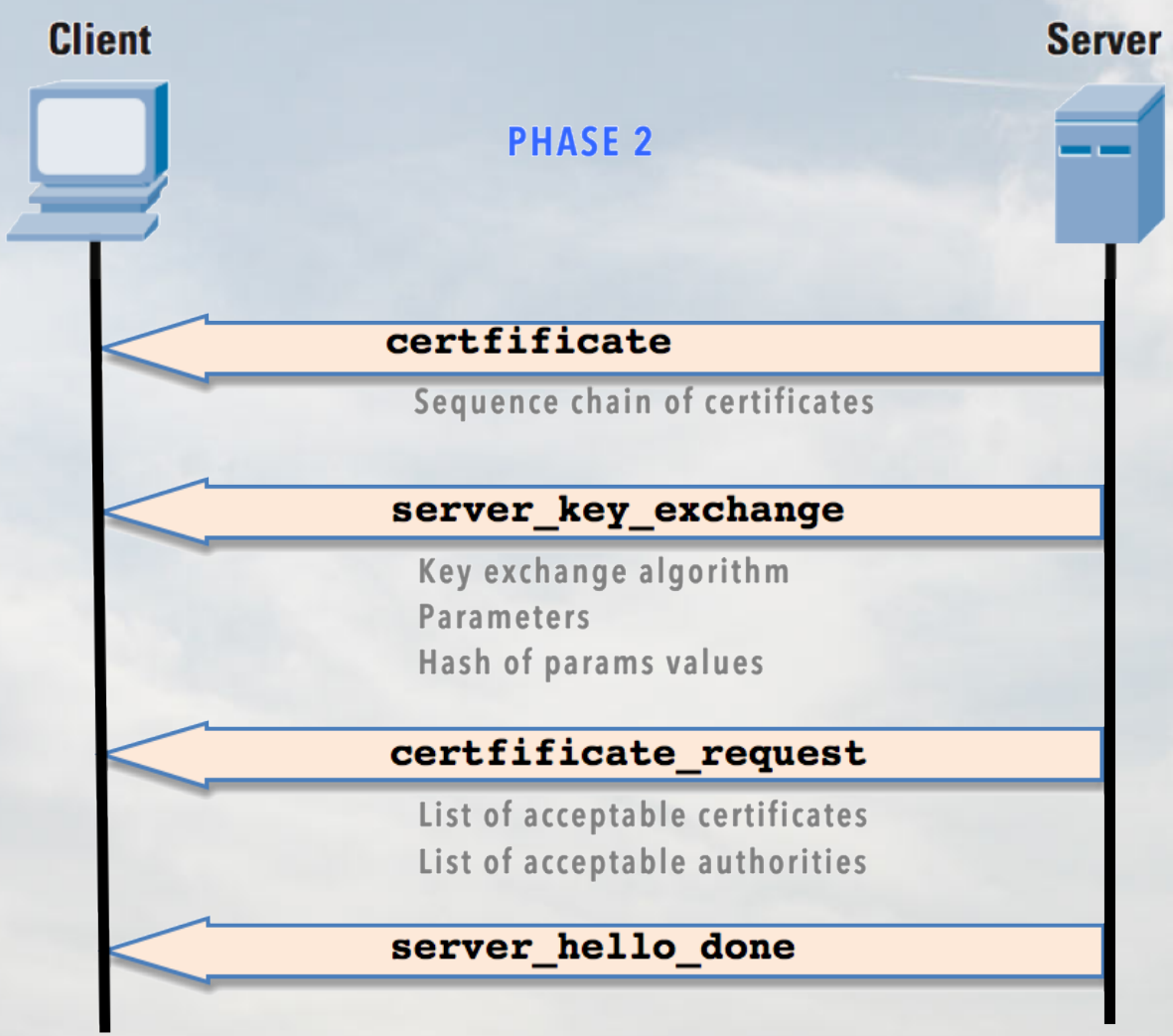



Client
Server
PHASE 1
Exchange security enhancement capabilities
PHASE 2
Server authentication, key exchange
PHASE 3
Cliente authentication, key exchange
PHASE 4
Finish handshake protocol

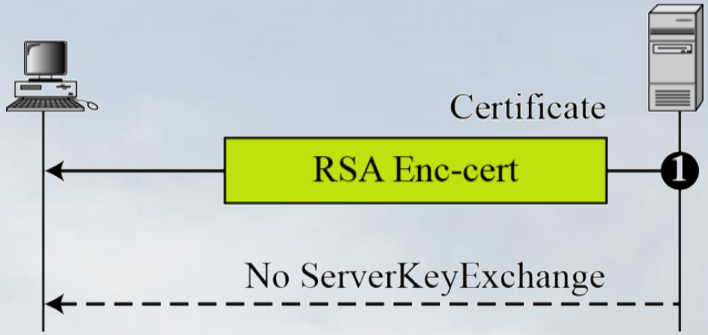
Establecimiento parámetros



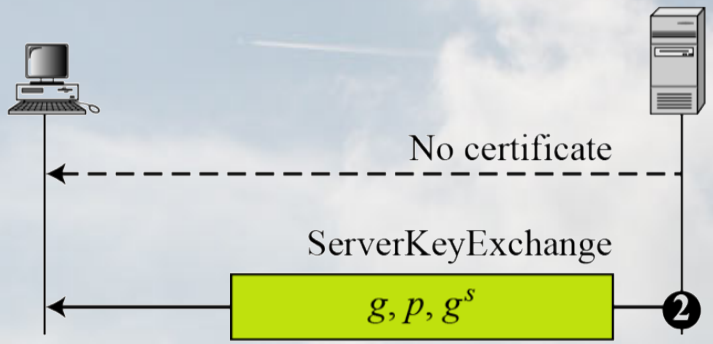
Autenticación del servidor



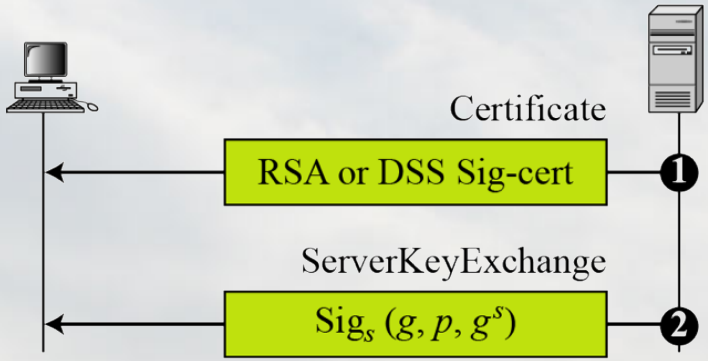
Intercambio de la *pre-master secret* (paso I)



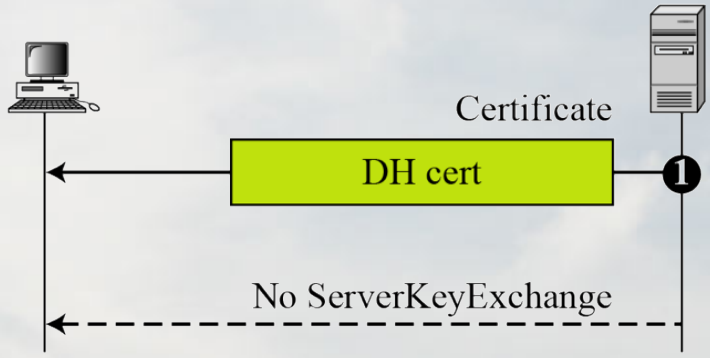
a. RSA



b. Anonymous DH



c. Ephemeral DH

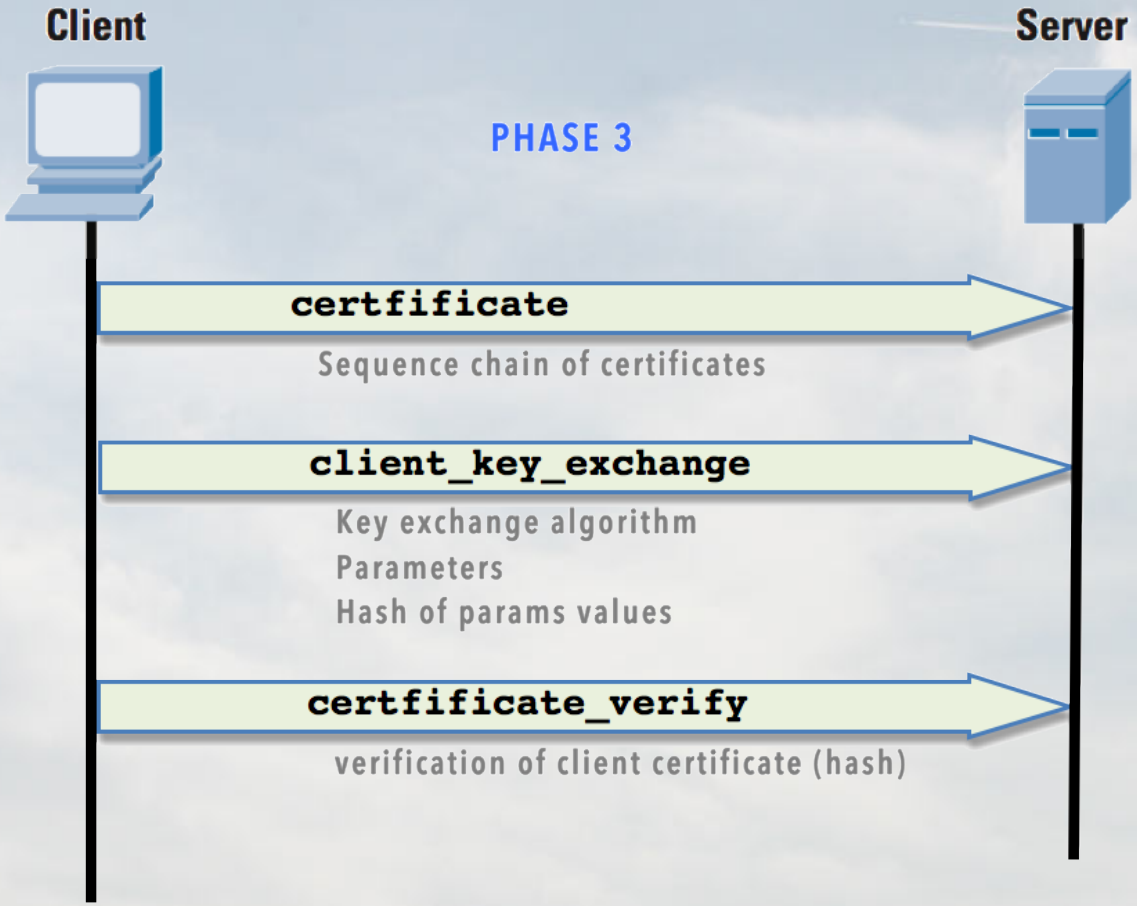


d. Fixed DH

fuente: web.engr.oregonstate.edu/~yavuza/



Autenticación del cliente

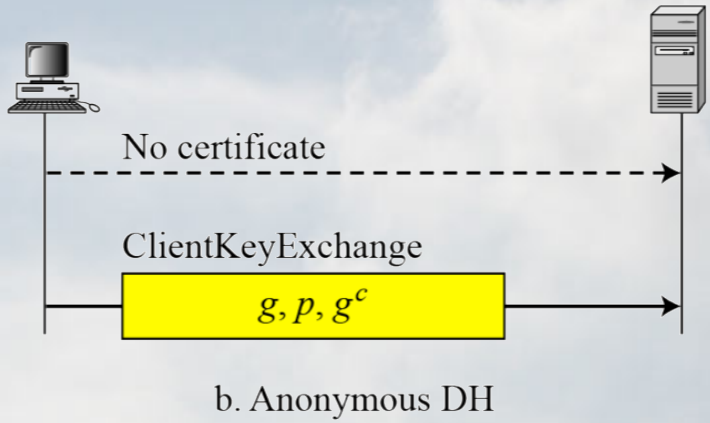


Intercambio de la *pre-master secret* (paso II)

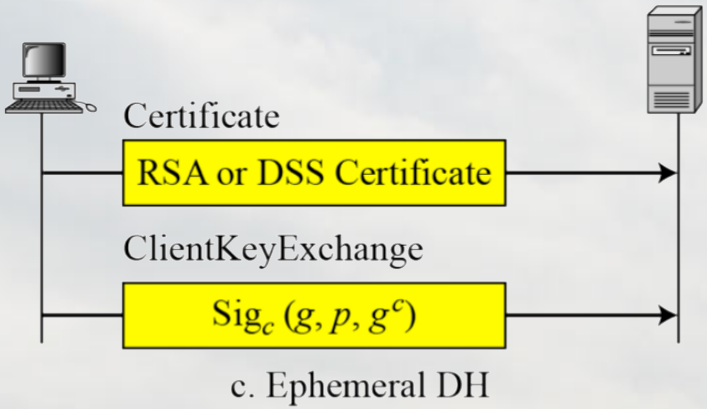
S: Encrypted with server's public key
Sig_c: Signed with client's public key



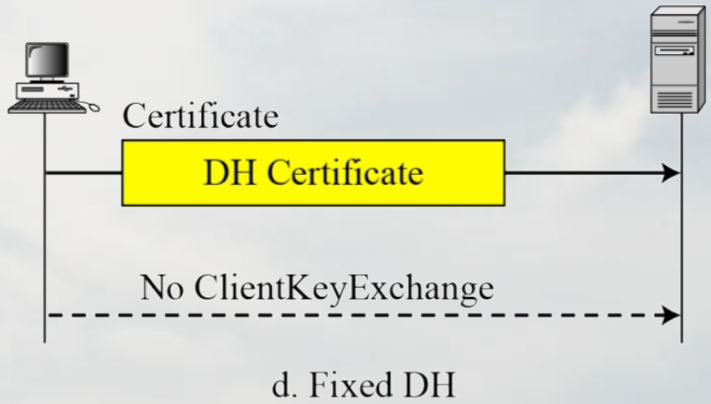
a. RSA



b. Anonymous DH



c. Ephemeral DH

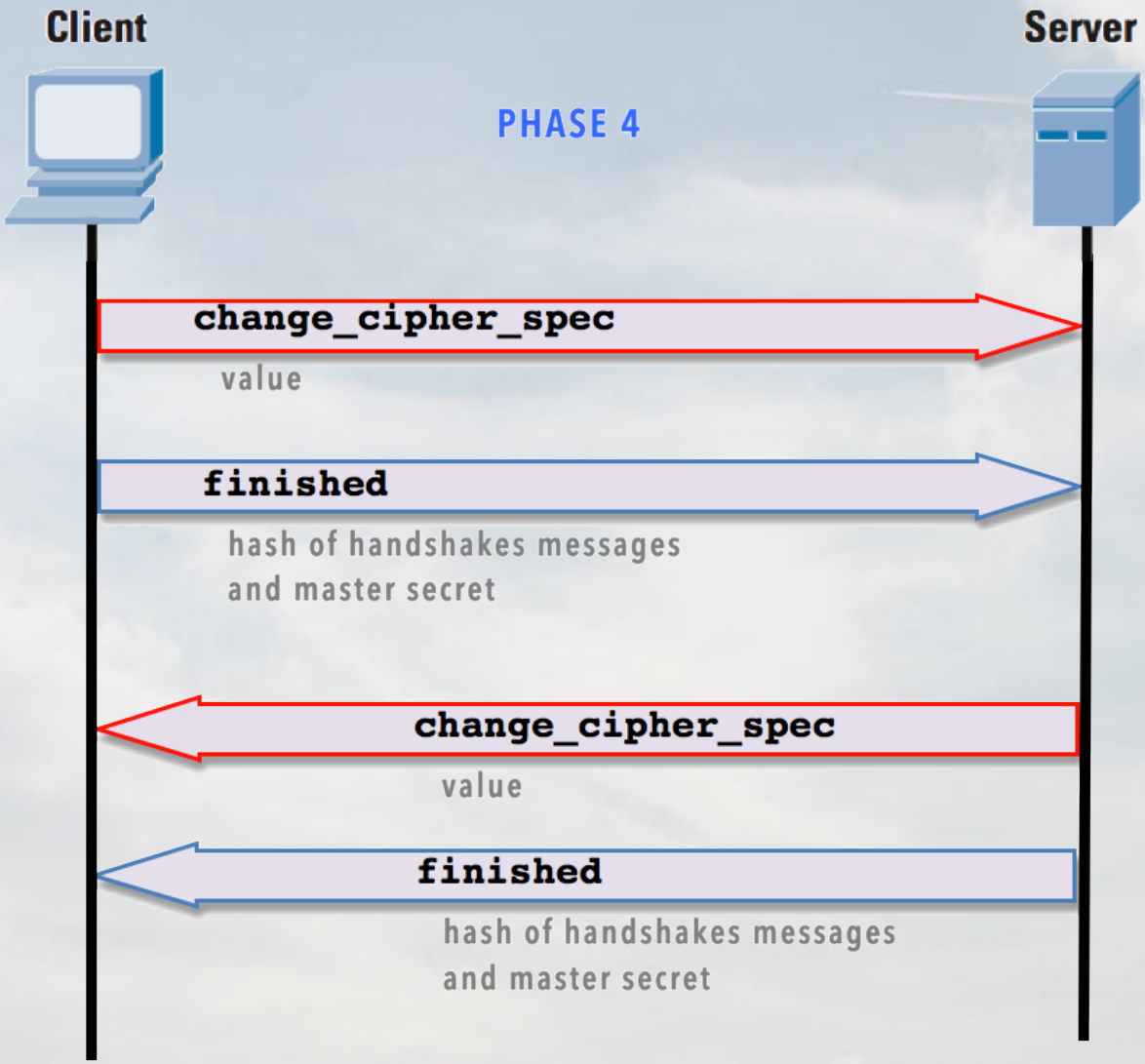


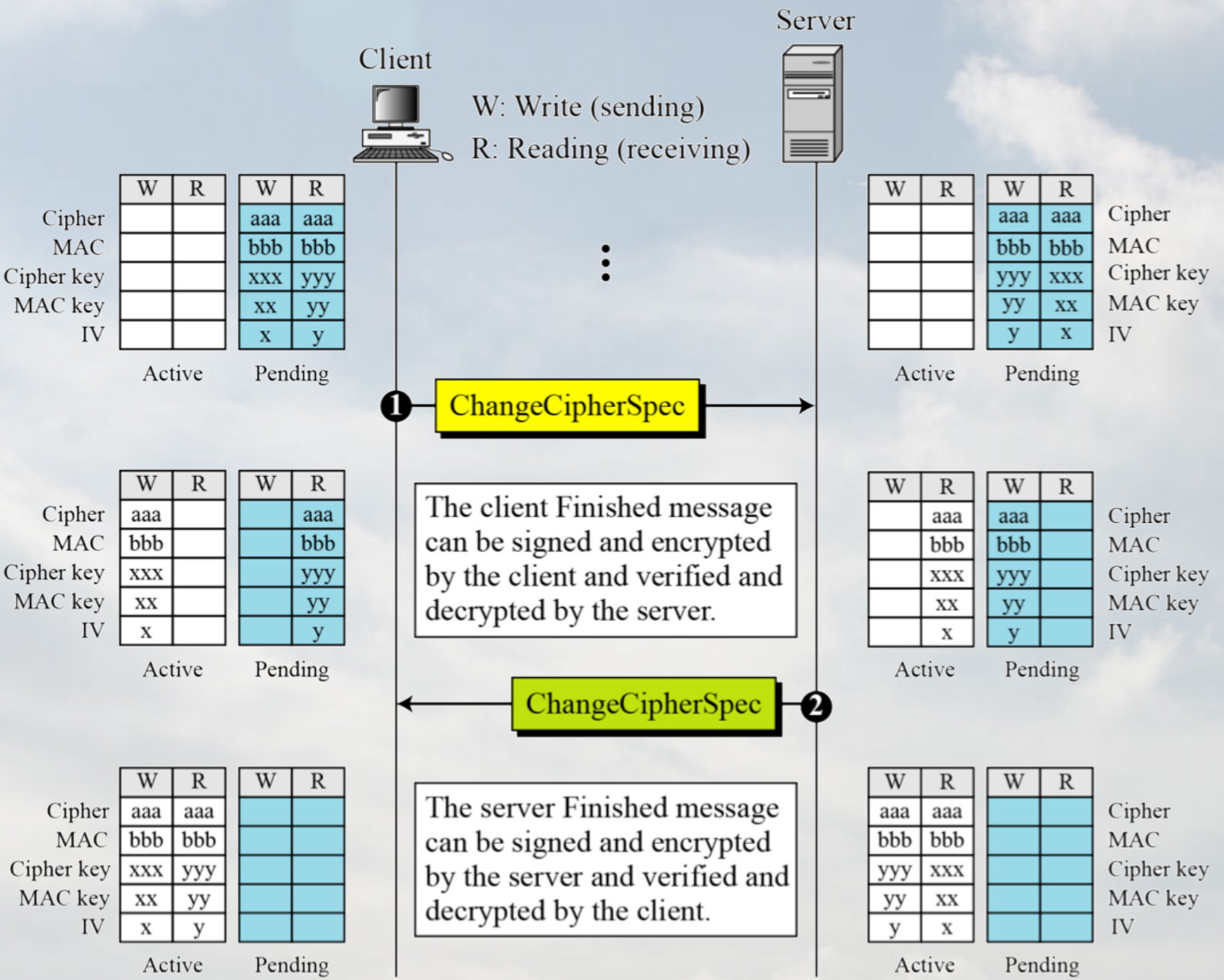
d. Fixed DH

fuente: web.engr.oregonstate.edu/~yavuza/

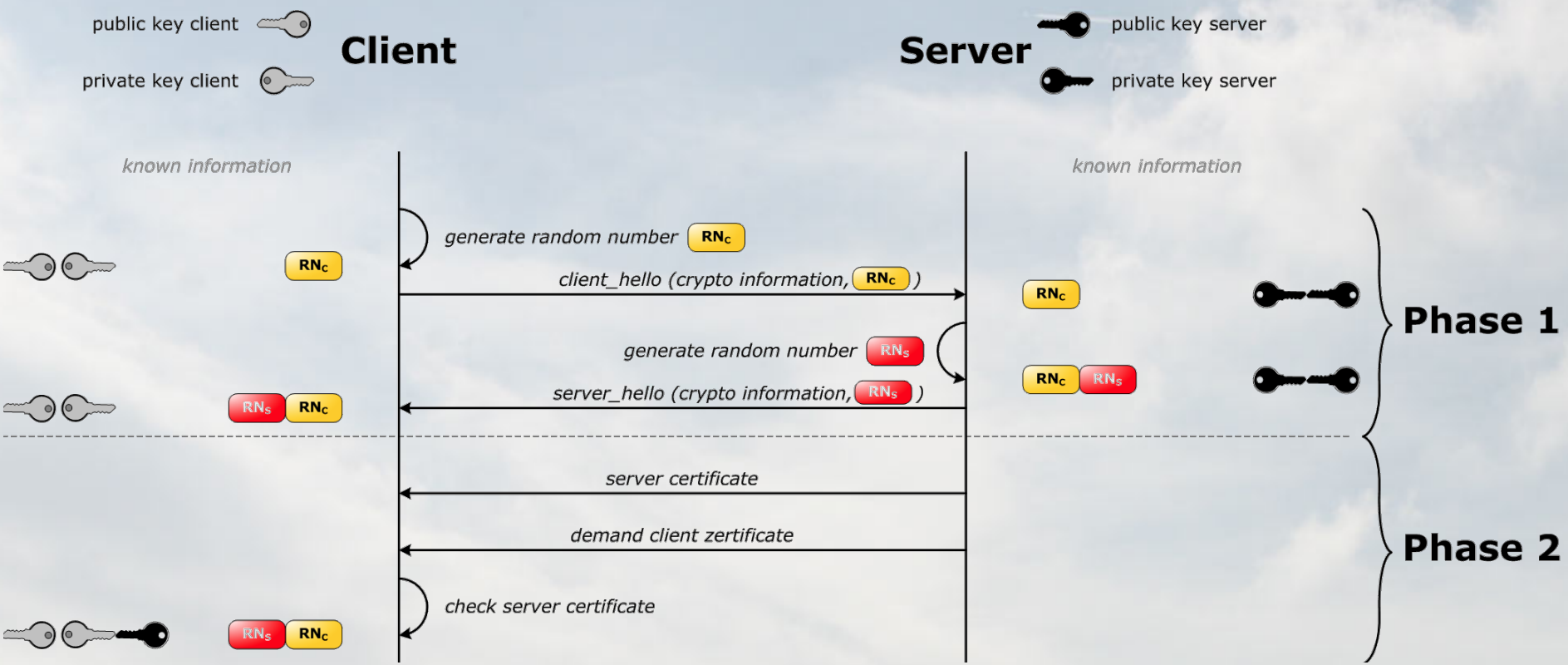


Finalización saludo

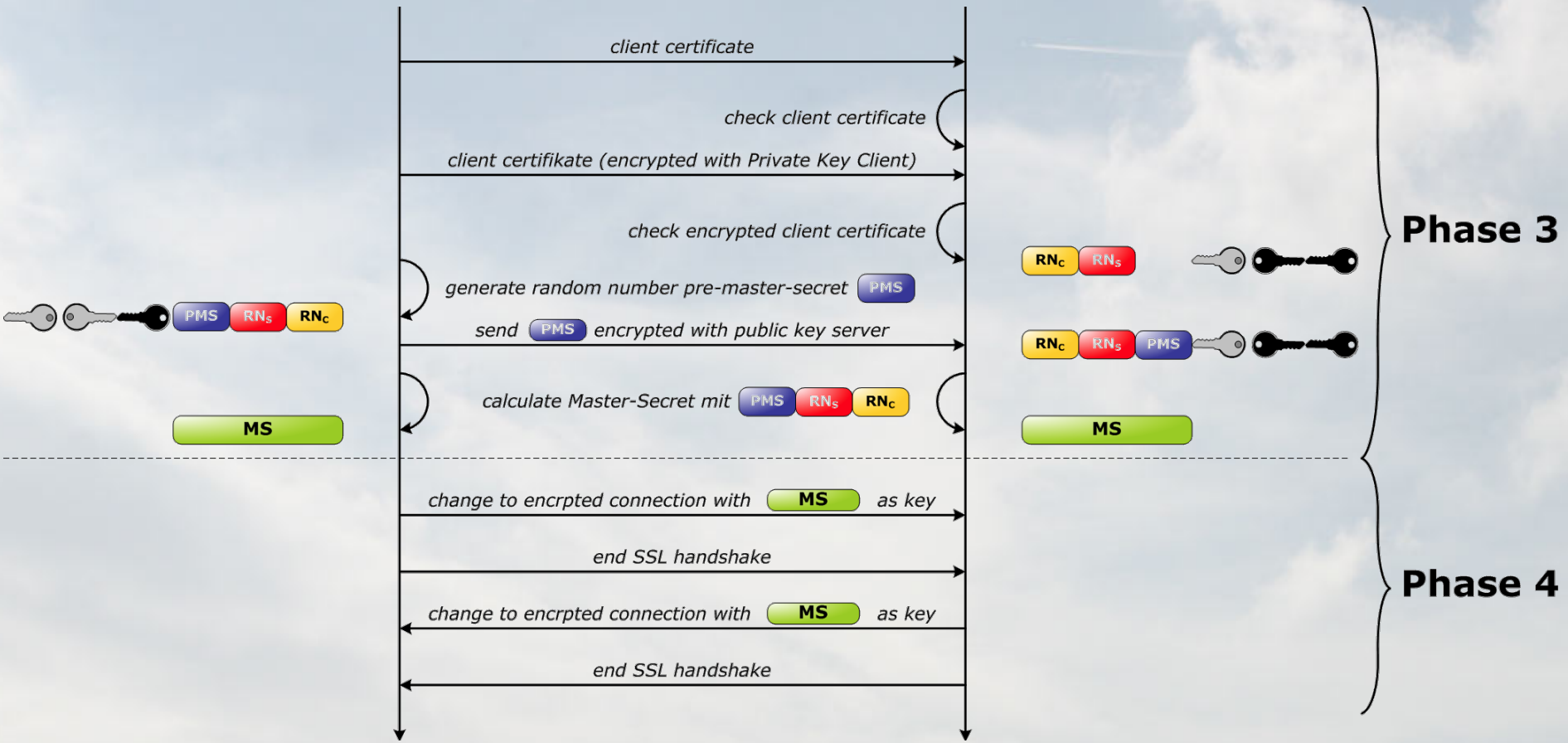




fuente: web.engr.oregonstate.edu/~yavuza/

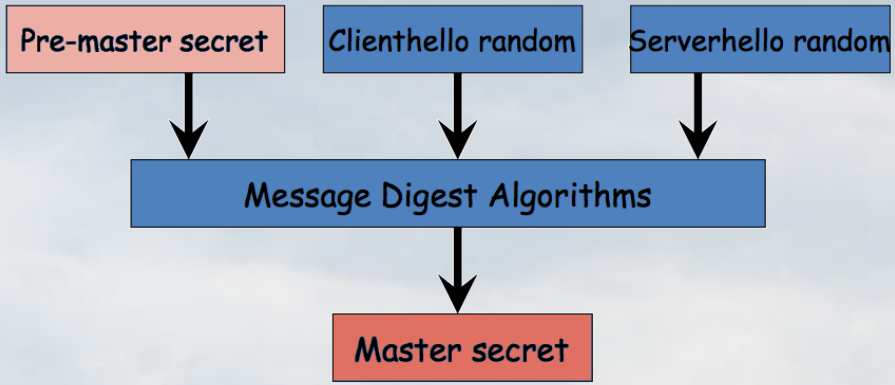


fuente: commons.wikimedia.org, Christian Friedrich

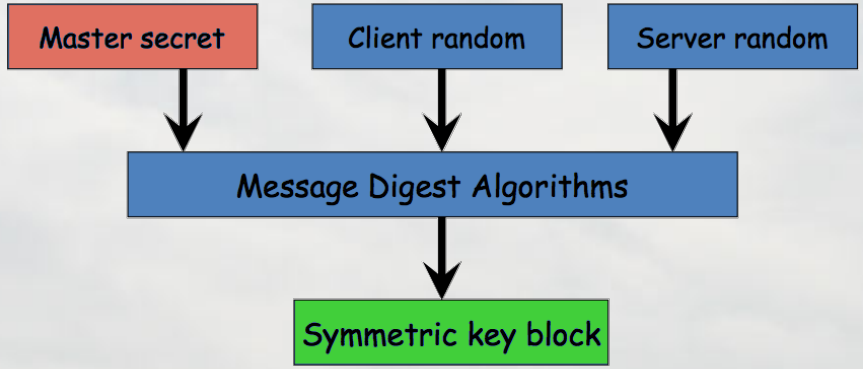


fuente: commons.wikimedia.org, Christian Friedrich

Generación parámetros criptográficos



(sesión)

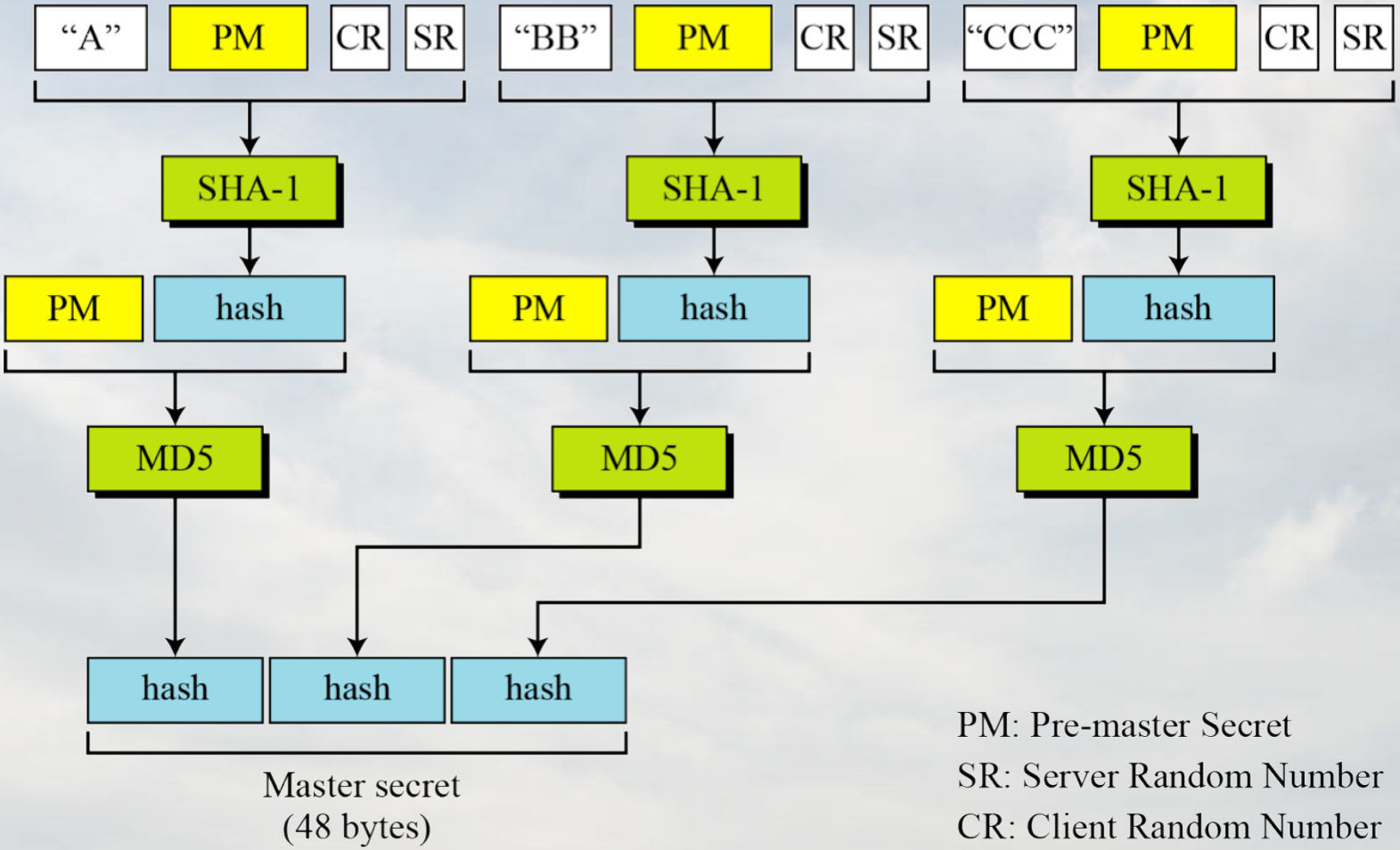


Symmetric key block = client write MAC secret, server write MAC secret, client write key, server write key, client write IV, and server write IV

(conexión)

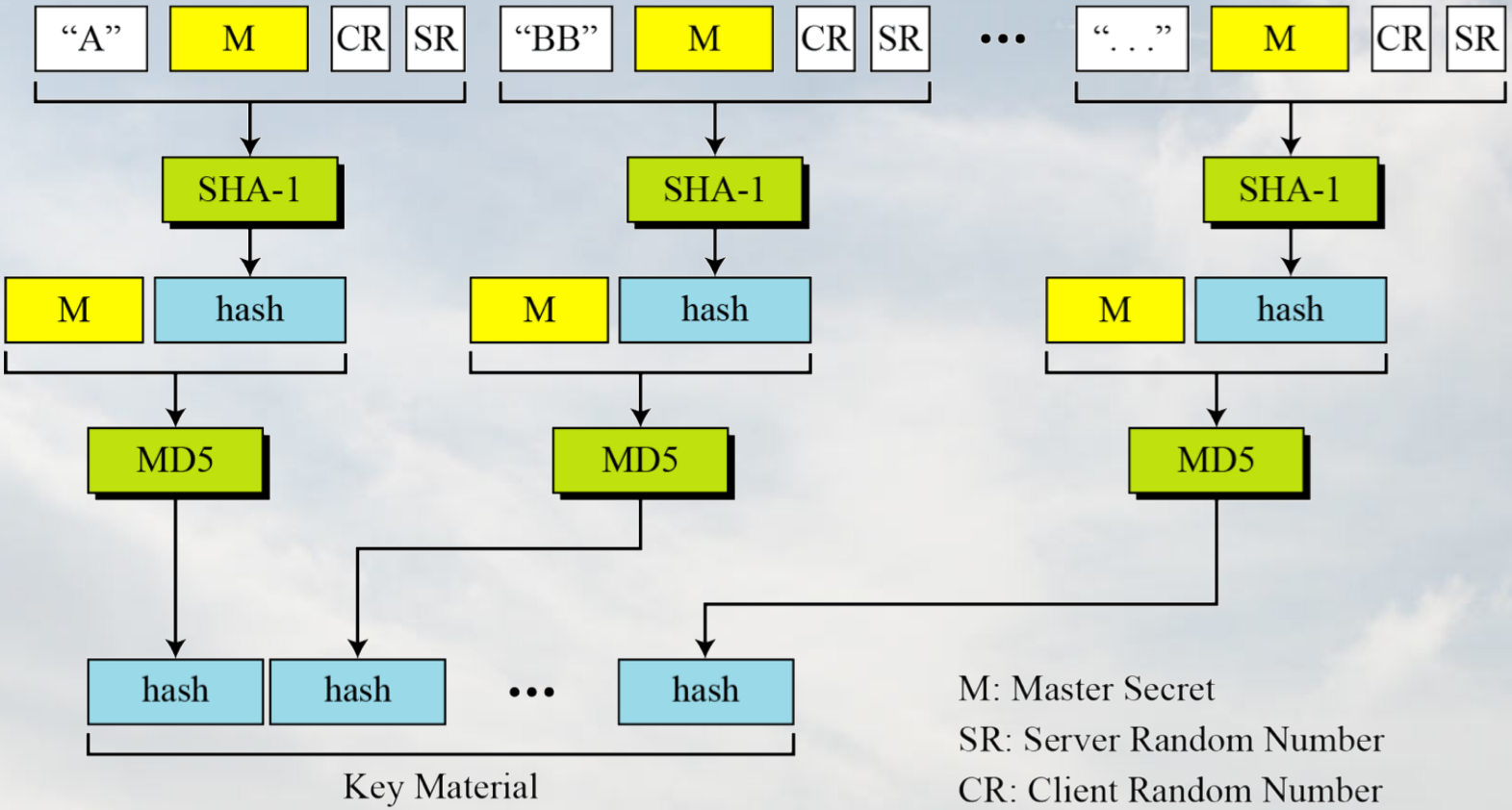
Detalle

Generar master secret



fuente: web.engr.oregonstate.edu/~yavuza/

Generar key material

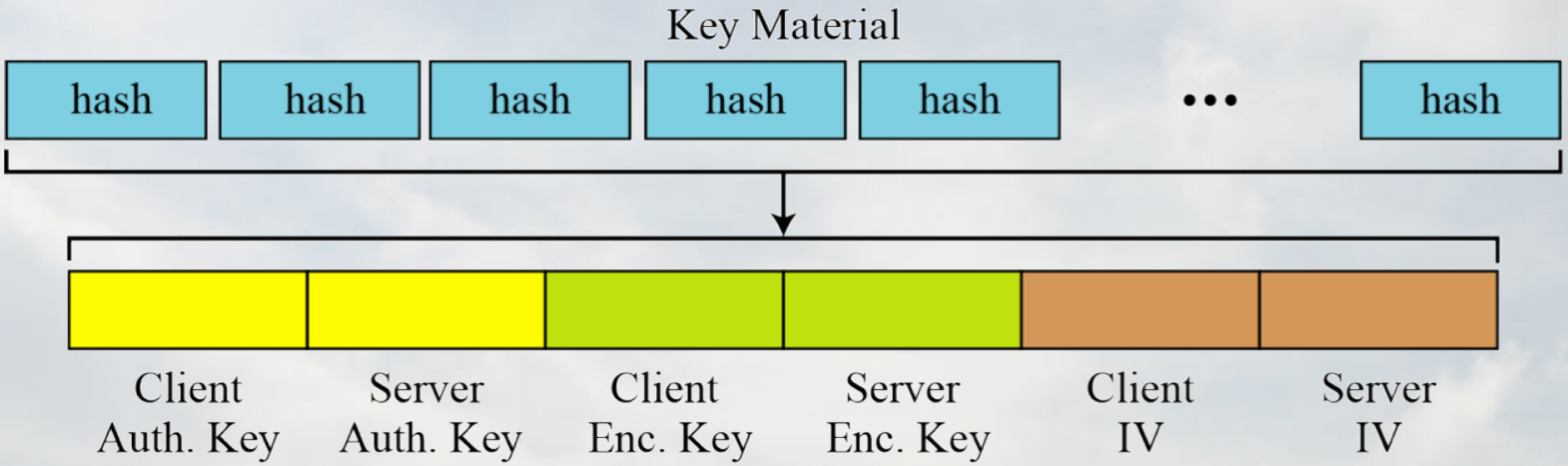


fuente: web.engr.oregonstate.edu/~yavuza/



Extraer parámetros criptográficos

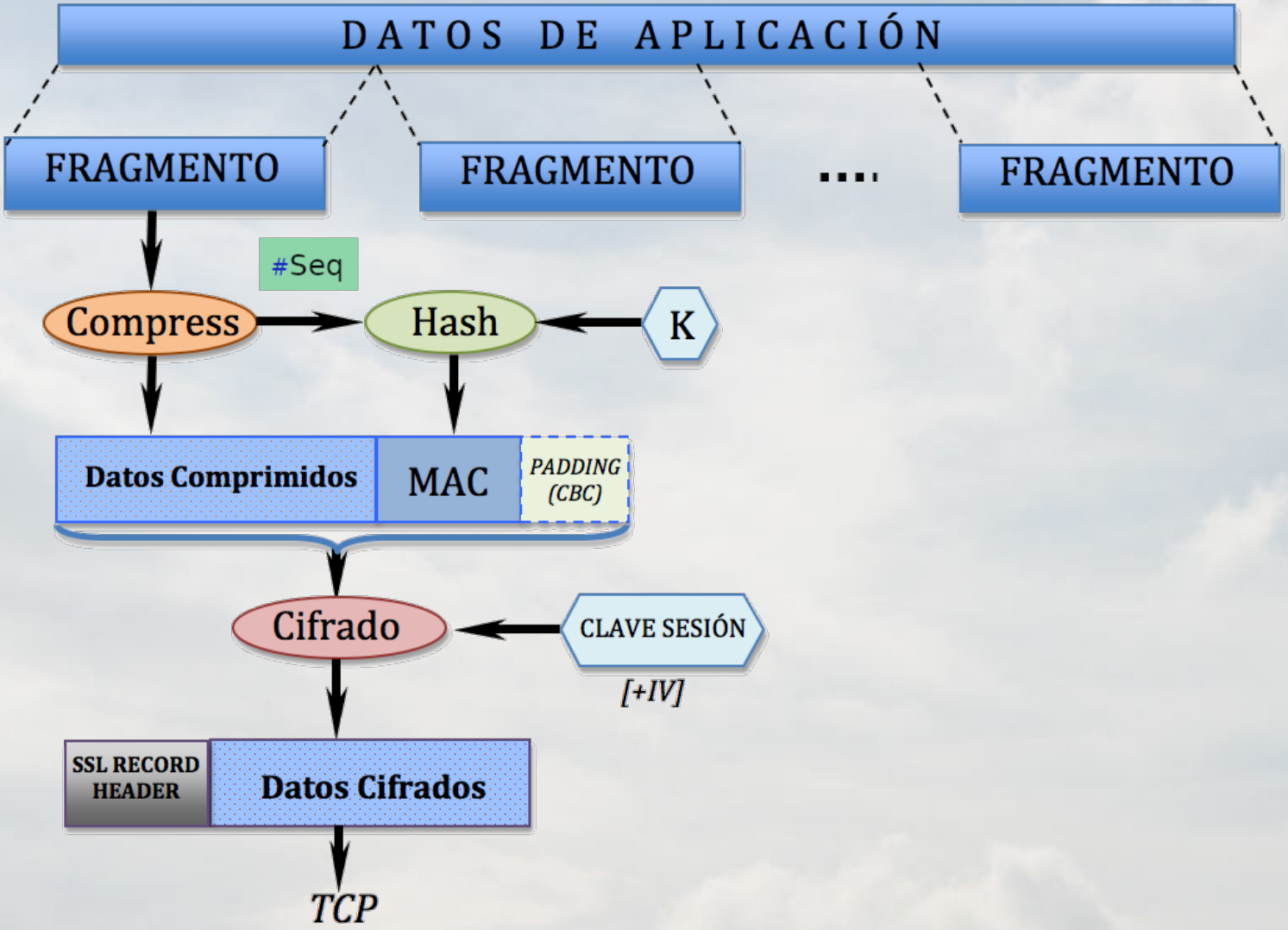
Auth. Key: Authentication Key
Enc. Key: Encryption Key
IV: Initialization Vector



fuente: web.engr.oregonstate.edu/~yavuza/

SSL Record Protocol

Especifica el modo de proteger y encapsular los datos a transmitir.



Formato mensajes

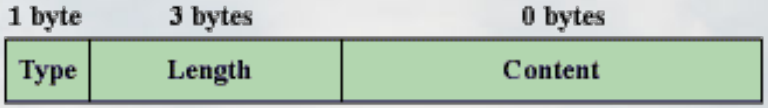
Record Protocol Header



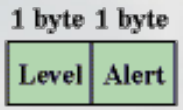
Payload (data protocol)



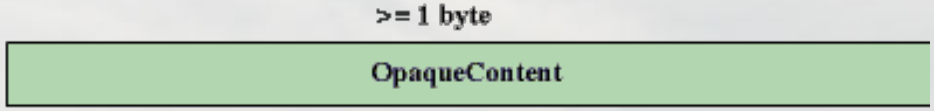
(a) Change Cipher Spec Protocol



(c) Handshake Protocol



(b) Alert Protocol



(d) Other Upper-Layer Protocol (e.g., HTTP)

fuente: williamstallings.com

SSL Change Cipher Spec Protocol

- Tras establecer los parámetros criptográficos durante la fase de saludo, se intercambian mensajes para conmutar el estado de estos parámetros de pendientes a activos para la conexión.
- Un único mensaje cifrado, que contiene un único byte con el valor 1.



SSL Alert Protocol

Se utiliza para notificación de alertas entre los pares (tras completar el *handshake*)

Mensajes de 2 bytes (Nivel + Alerta)

- Nivel: gravedad del mensaje, aviso (1) o fatal (2)
- Alerta: código que indica la alerta específica:
  - Mensaje inesperado (10)
  - MAC incorrecto (20)
  - Error al descomprimir datos (30)
  - Certificados revocados (44)
  - ...



Seguridad

Vulnerabilidades críticas sobre SSLv3

- Freak attack(Mz15) afecta algunas implementaciones.
- Heartbleed(Mz14) afecta a la librería OpenSSL usada para implementar SSL/TLS
- Poodle(Oc14) afecta a los cifrados CBC y su relleno
- RFC 6176: Prohibiting Secure Sockets Layer (SSL) Version 2.0(Mz11)
- ...

RFC 7568: Deprecating Secure Sockets Layer Version 3.0

testssl.sh



Transport Layer Secure v1.2 (RFC 5246-*obsolete*)

- Estandarización de la IEFT basada en SSL (SSLv3 $\Longrightarrow$ TLS v1.0)
- Más seguro y eficiente
- Cambios:
  - *Cipher suite*, descarta métodos con DES e IDEA (con Fortezza, en v1.0)
  - Soporte a SHA-256, PSK y ECC
  - Cipher-suite `TLS_RSA_WITH_AES_128_CBC_SHA` es obligatoria
  - Authenticated encryption: CCM y GCM (RFC 5116)
  - Algoritmos de compresión NULL y DEFLATE (RFC 3749)
  - Pseudo-random function (PRF) para la generación de `master-key`
  - *Record Protocol*, actualizada versión y diferencias en los esquemas MAC
  - *Handshake protocol*, cálculo diferente de los mensajes *Certificate-verify* y *Finished (PRF)*
  - *Alert Protocol*, más códigos adicionales(RFC5246)
  - Exported keying material (EKM) to mitigate MITM attacks (RFC 5705)



Illustrated TLS Connection

Extensiones TLS (RFC 6066)

- El mayor cambio es el mecanismo de extensión que permite incorporar funciones adicionales en el TLS sin tener que cambiar el protocolo subyacente
- Se incorporan/negocian en los mensajes `ClientHello/ServerHello`
  - Cada extensión comprende: un campo *Type* (2B), un campo *length* (2B) y un campo de datos (long. variable)
- Extensiones más comunes:
  - `server_name` (*Server Name Indication*-SNI)
  - `status_request` (OCSP stapling)
  - `signature_algorithms` (algoritmos firma/hash)
  - `heartbeat` (keep-alive)
  - `signed_certificate_timestamp` (*transport of the SCT for CT*)
  - `session_ticket` (*session resumption* - RFC 5077)
  - `elliptic_curves` (RFC 4492)
  - ...



Generación parámetros criptográficos (*Key derivation*)

$A(0) = seed$
$A(i) = HMAC_hash\,(secret, A(i-1))$
Expansion Function (hash=SHA256):
$P_hash\,(secret, seed) = \quad HMAC_hash\,(secret, A(1)\,||\,seed)\;||$
$HMAC_hash\,(secret, A(2)\,||\,seed)\;||\;\ldots$

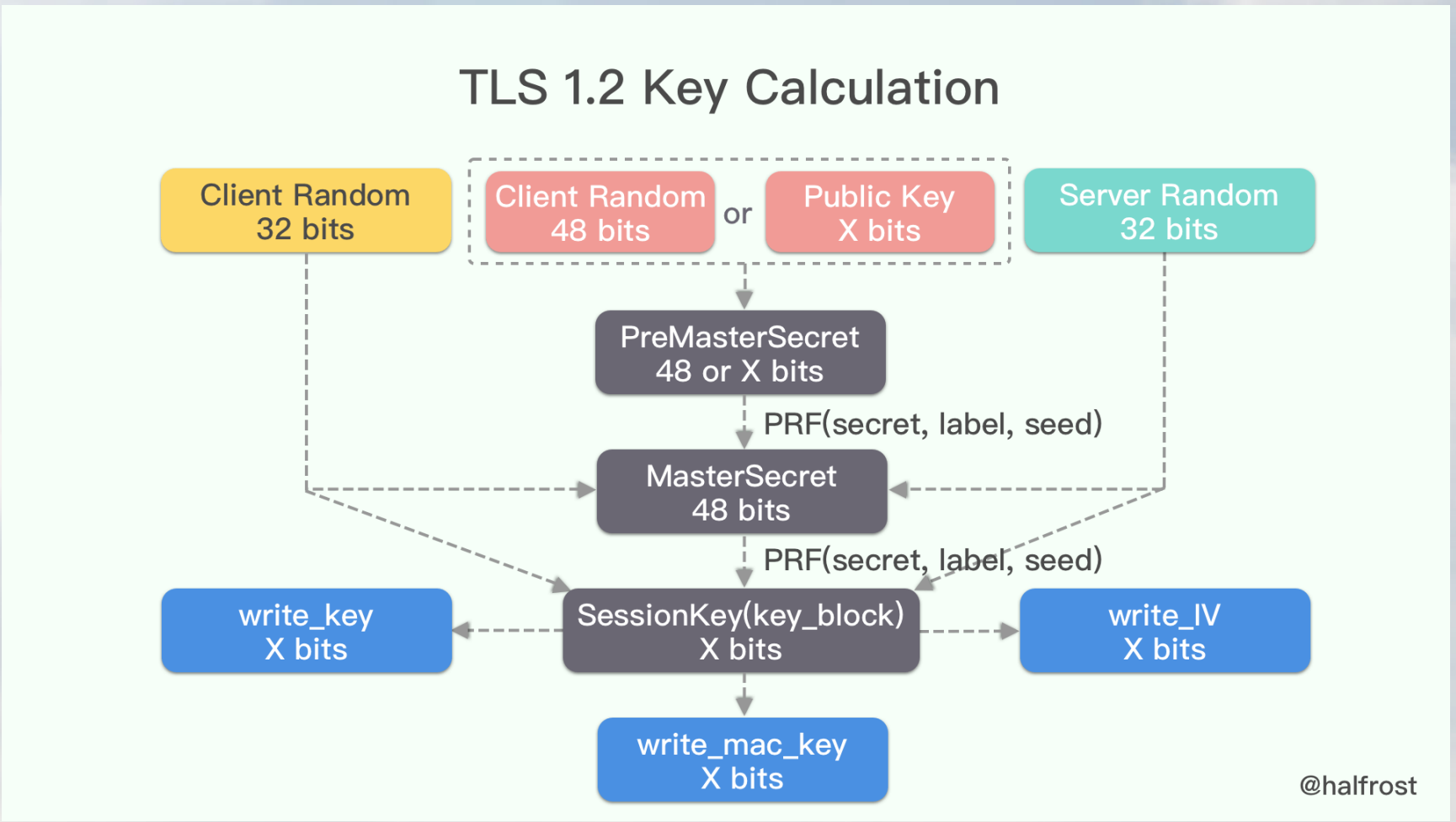
Pseudo-Random Function:
$PRF\,(secret, label, seed) = P_\,\langle hash\rangle\,(secret, label\,||\,seed)$

Key calculation:
$master_secret = PRF(premaster_secret, "master\ secret",$
$ClientHello.random\,||\,ServerHello.random)$

$key_block = PRF(master_secret, "key\ expansion",$
$ServerHello.random\,||\,ClientHello.random)$

$key_block \Longrightarrow [KeyMAC-C\,|\,KeyMAC-S\,|$
$|KeyEncrypt-C\,|\,KeyEncrypt-S\,|\,VI-C\,|\,VI-S]$





source: https://github.com/halfrost/Halfrost-Field/blob/master/contents/Protocol/HTTPS-key-cipher.md



TLS Session Resumption

Retomar una sesión previa con un servidor (o establecer conexiones paralelas) utilizando parámetros criptográficos de una sesión anterior, reduciendo el handshake inicial

Dos mecanismos:

- **Session ID** (RFC 5246)
  Servidor envía mensaje "*Server Hello*" que incluye un "*Session ID*"
  El cliente lo guarda y lo enviará en su próximo "*Client Hello*"
  Si aún está en caché, el servidor retomará la sesión con un *SSL handshake* abreviado (*session resumption*).
  Si no, emitirá un "*new Session ID*" y comenzará un *SSL handshake* completo (*session renegotiation*).
  No es escalable.



- **Session Ticket** (RFC 5077)
  El cliente indica que soporta este mecanismo mediante una extension (*SessionTicket*) en el mensaje "Client Hello"
  Al final de un *TLS handshake* completo (antes de *ChangeCipherSpec*), el servidor incluye un mensaje "*New Session Ticket*" que contiene el "*session state*" completo, cifrado y protegida su integridad por una clave solo conocida por el servidor.
  El cliente retomará una sesión enviando la extensión "*session ticket*" al servidor.



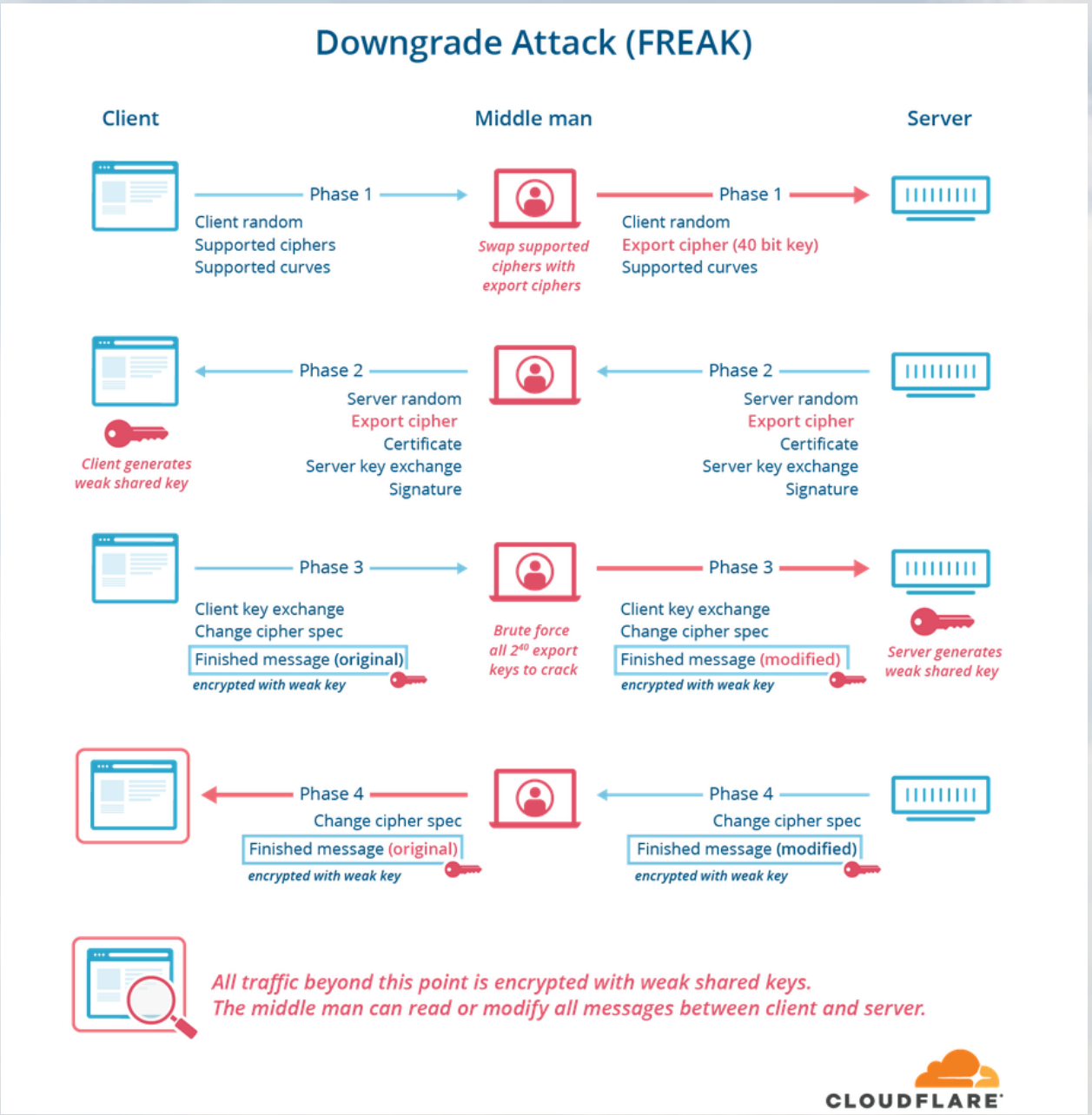
Seguridad

Vulnerabilidades y ataques sobre TLSv1.2:

- RSA no proporciona PFS
- Cifrados EXPORT (RSA_EXPORT): ataques FREAK and LogJam (CVE-2015-0204)
- Cifrado de flujo RC4, no es seguro (CVE-2015-2808)
- Grupos DH arbitrarios (CVE-2016-0701)
- Cifrado en modo CBC (CVE-2016-2183): ataques BEAST y Lucky13
- Several TLS implementations did not properly validate the padding bytes: POODLE TLS and Cache-like Attacks
- *Renegotiation attack* (CVE-2009-3555)
- Colisiones en SHA-1, en MD5

RFC 7457(Fb15): *Summarizing Known Attacks on TLS and DTLS*
RFC 7525 (My15): *Recommendations for Secure Use of TLS and DTLS*
Monitoring the quality of SSL/TLS support
Seguridad del protocolo SSL/TLS




Downgrade Attack (FREAK)
Client
Middle man
Server
Phase 1
Client random
Supported ciphers
Supported curves
Swap supported ciphers with export ciphers
Phase 1
Client random
Export cipher (40 bit key)
Supported curves
Phase 2
Server random
Export cipher
Certificate
Server key exchange
Signature
Client generates weak shared key
Phase 2
Server random
Export cipher
Certificate
Server key exchange
Signature
Phase 3
Client key exchange
Change cipher spec
Finished message (original)
encrypted with weak key
Brute force all 2^40 export keys to crack
Phase 3
Client key exchange
Change cipher spec
Finished message (modified)
encrypted with weak key
Server generates weak shared key
Phase 4
Change cipher spec
Finished message (original)
encrypted with weak key
Phase 4
Change cipher spec
Finished message (modified)
encrypted with weak key
All traffic beyond this point is encrypted with weak shared keys.
The middle man can read or modify all messages between client and server.
CLOUDFLARE

Transport Layer Secure v1.3 (RFC 9846)

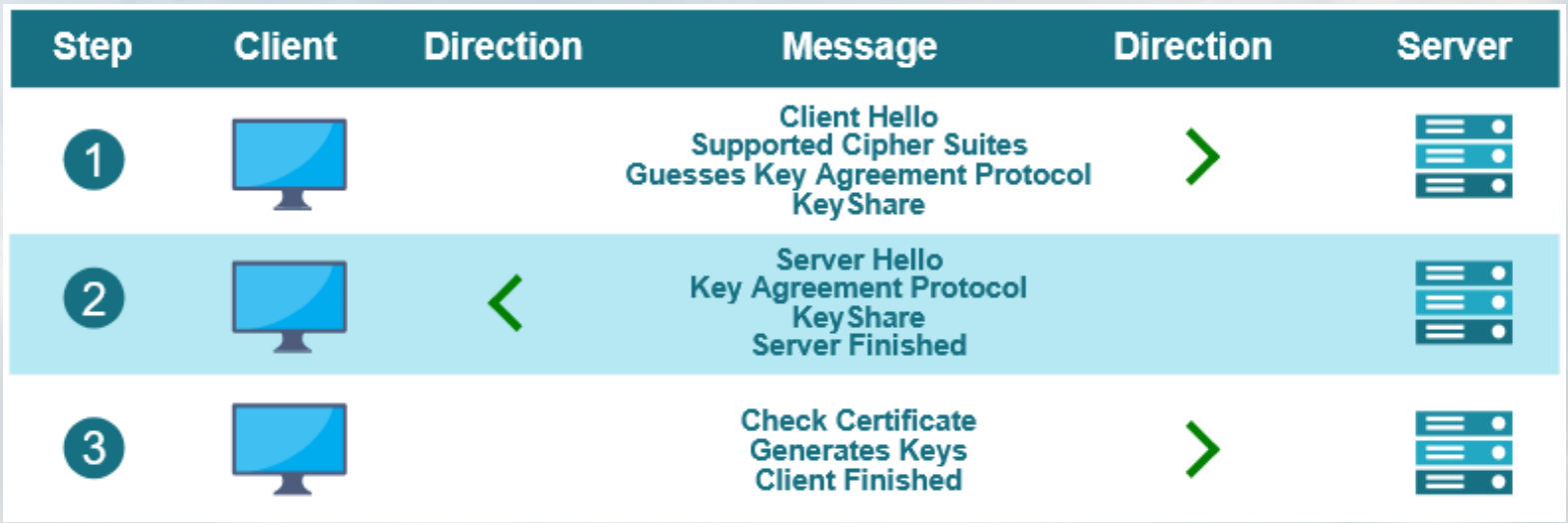
RFC 9846: "supersedes and obsoletes previous versions of TLS" (Jl 26)

- *Backward Compatibility*: implementaciones en TLS 1.3 pueden conectarse a TLS 1.2
  - Algunos campos en los mensajes desestimados en TLS 1.3 se mantienen en modo *legacy,* por ejemplo, *sessionID*
  - *Change Cipher spec protocol*, *deprecated*. Se preserva por compatibilidad hacia atrás.
  - Compresión siempre NULL.



Handshake protocol

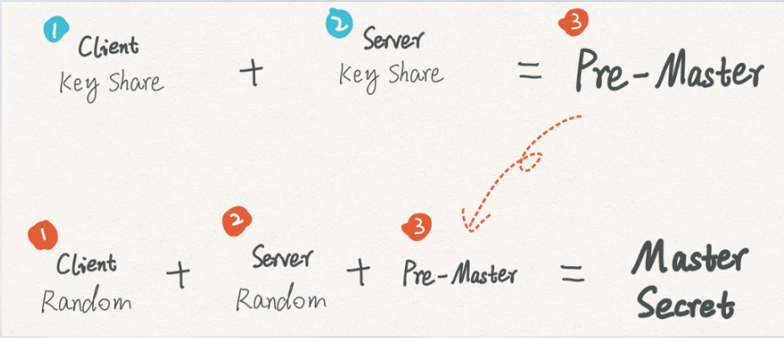
Conexiones se establecen en 1-RTT *:*



fuente: https://aboutssl.org/ssl-tls-handshake-overview/

Mejora la seguridad y la eficiencia (reduciendo latencia):

- Clientes comienzan intercambio de claves en *ClientHello*, propone uno o más métodos *Key-exchange* y envía los parámetros necesarios mediante la extension "*key_share*"
- Servidor puede calcular *master-secret* (*main-secret*)



fuente: https://cabulous.medium.com/tls-1-2-andtls-1-3-handshake-walkthrough-4cfd0a798164

Si el servidor no acepta, envía *HelloRetryRequest*

- En el *handshake*, los mensajes tras *ServerHello* están cifrados. Y el servidor **firma todos** los mensajes del *handshake*.
- No admite renegociación, en una sesión segura ya establecida renegociar nuevos parámetros criptográficos o para solicitar certificado al cliente. Reemplazado por a *"lightweight key update"* y *"post-handshake client authentication"*.



- ***Session resumption***, servidor envía una *PSK identity* derivada del handshake inicial mediante un mensaje cifrado *post-handshake: NewSessionTicket*
  - Conexiones paralelas: servidor envía varios tickets para que cada nueva conexión pueda usar uno distinto y único (RFC 9149).
  - Cliente utiliza la PSK de un ticket para realizar un *handshake* parcial (sin certificados) que solo negocia las claves nuevas para el resto de la conexión.
- Conexiones en **0-RTT** (*zero round-trip time mode*) con `EarlyData`
  - Cliente utiliza una PSK obtenida de una sesión anterior (*NewSessionTicket*), cifrando los datos del primer mensaje (*EarlyData*) con las claves derivadas de la PSK
  - Negocia claves nuevas para el resto de la conexión.



- *Ciphersuite*, negociación por separado:
  - *Key exchange:* en extensiones
    - ★ *StaticRSA* y *Diffie-Hellman* descartados
    - ★ Soporta: (EC)DHE, PSK-only, PSK-with-(EC)DHE
  - *Authentication*: algoritmos para validar firmas en extensiones
  - *Key derivation*: Función rediseñada –> HMAC-based Key Derivation Function (HKDF)
  - *Nuevo formato: AEAD cipher + Hash function for HKDF*

TLS_AES_128_GCM_SHA256

Protocol	AEAD	Bits	Cipher	HMAC-KDF
TLS			Mode	

No soporta algoritmos *legacy* de cifrado simétrico

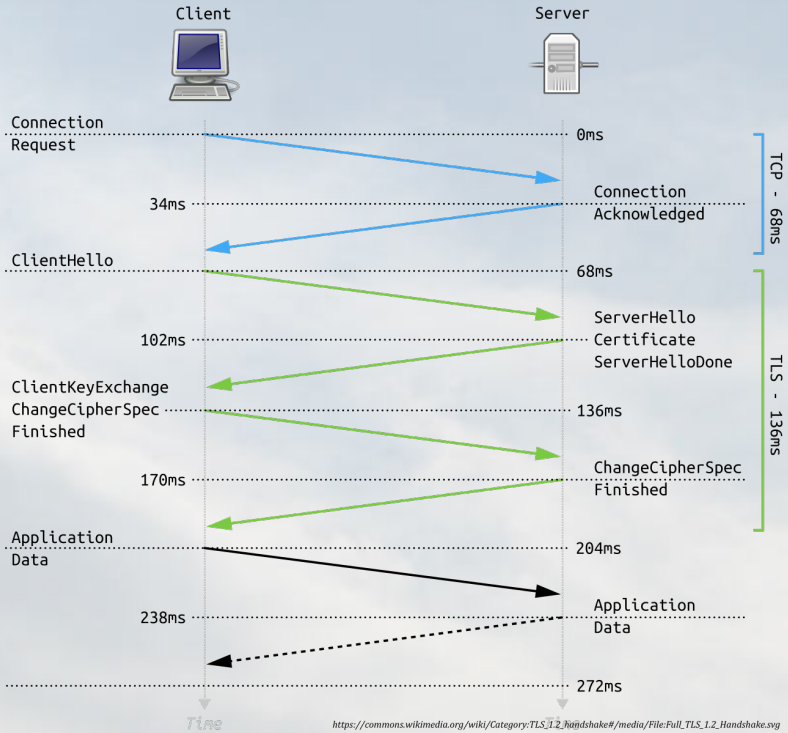


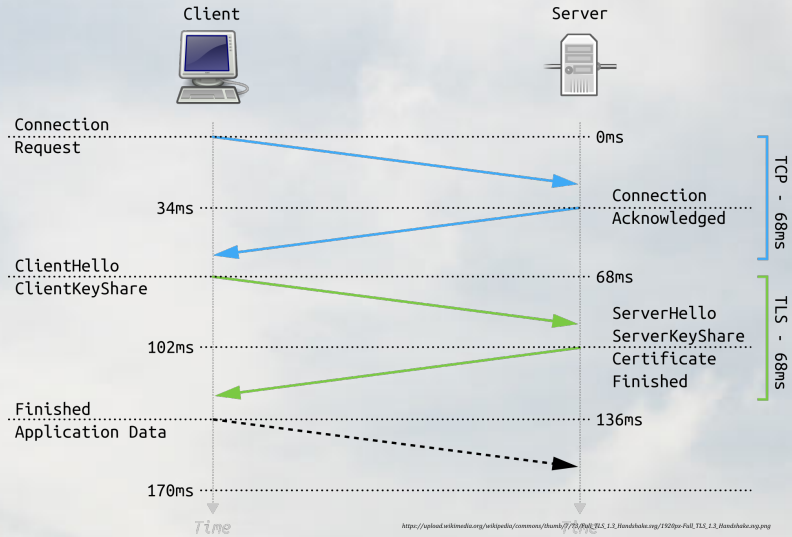
- Sesiones seguras no se consideran establecidas hasta que haya concluido el *handshake* (principal)
- Incrementa el tipo de mensajes que soportan extensiones
- *TLS Encrypted Client Hello extension (RFC 9849)*
- Cambios en el modo de transportar mensajes OCSP
- Negociar protocolo de aplicación (ALPN extension - RFC 7301)
- ***Record Protocol***, se ocultan (*ApplicationData*) los tipos de mensajes cifrados por privacidad

OpenSSL 3.5 incluye la implementación de TLSv1.3

TLS1.3 *Illustrated TLS Connection*

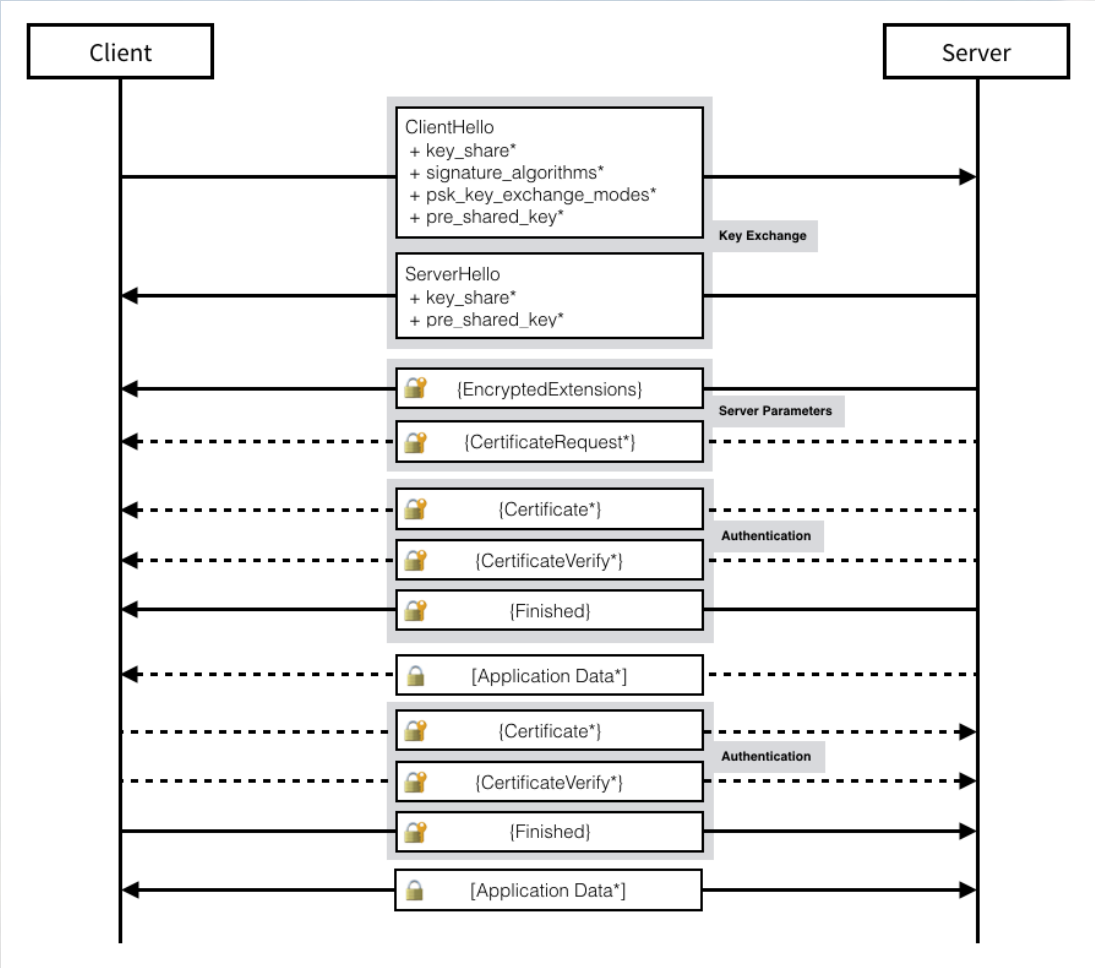



Client
Server
Connection Request
0ms
34ms
Connection Acknowledged
TCP - 68ms
ClientHello
68ms
ServerHello
Certificate
ServerHelloDone
102ms
ClientKeyExchange
ChangeCipherSpec
Finished
136ms
TLS - 136ms
ChangeCipherSpec
Finished
170ms
Application Data
204ms
Application Data
238ms
272ms
Time
https://commons.wikimedia.org/wiki/Category:TLS_1.2_handshake#/media/File:Full_TLS_1.2_Handshake.svg


Client
Server
Connection Request
0ms
34ms
Connection Acknowledged
TCP - 68ms
ClientHello
ClientKeyShare
68ms
ServerHello
ServerKeyShare
Certificate
Finished
102ms
TLS - 68ms
Finished
Application Data
136ms
170ms
Time

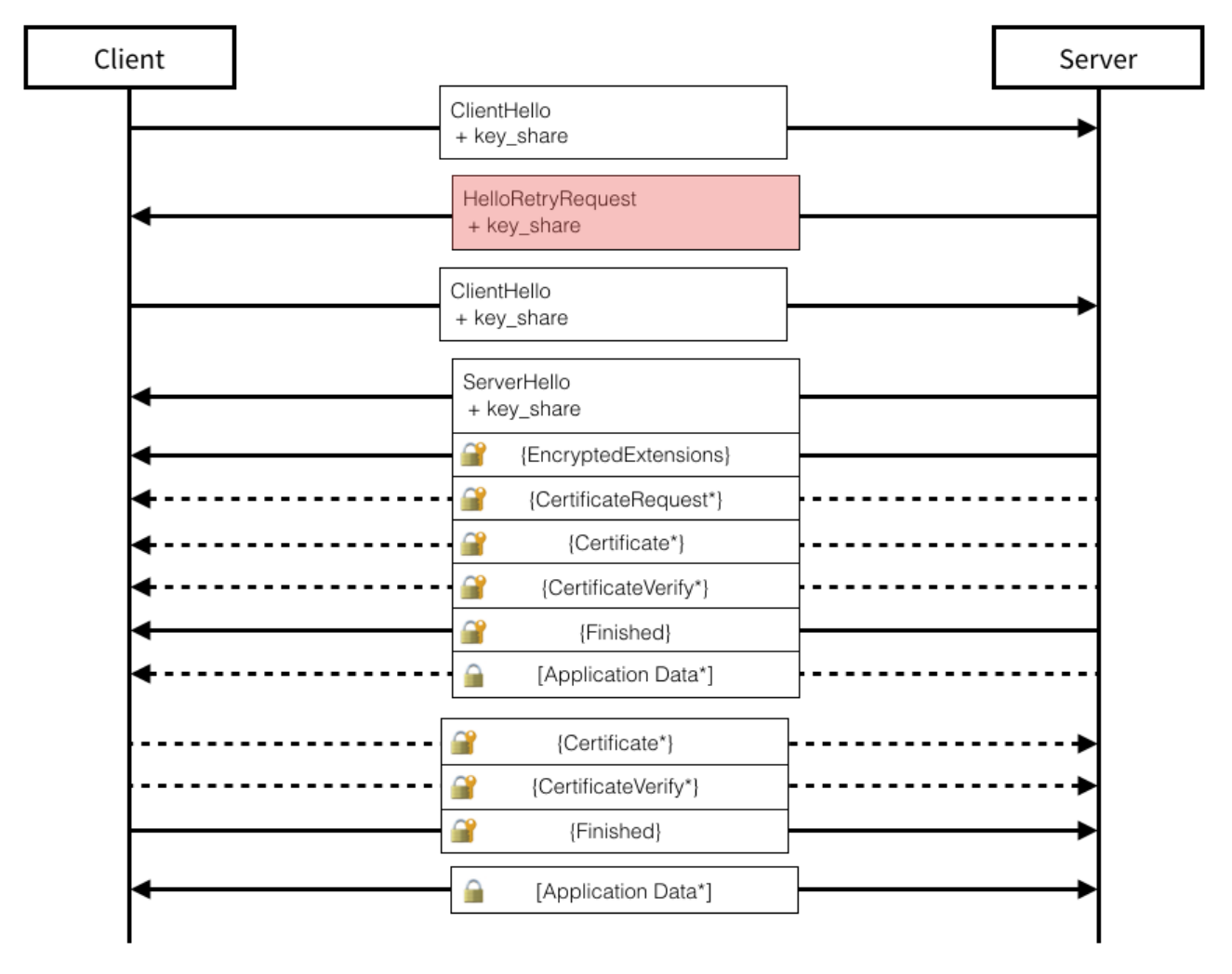
en detalle

Full handshake:



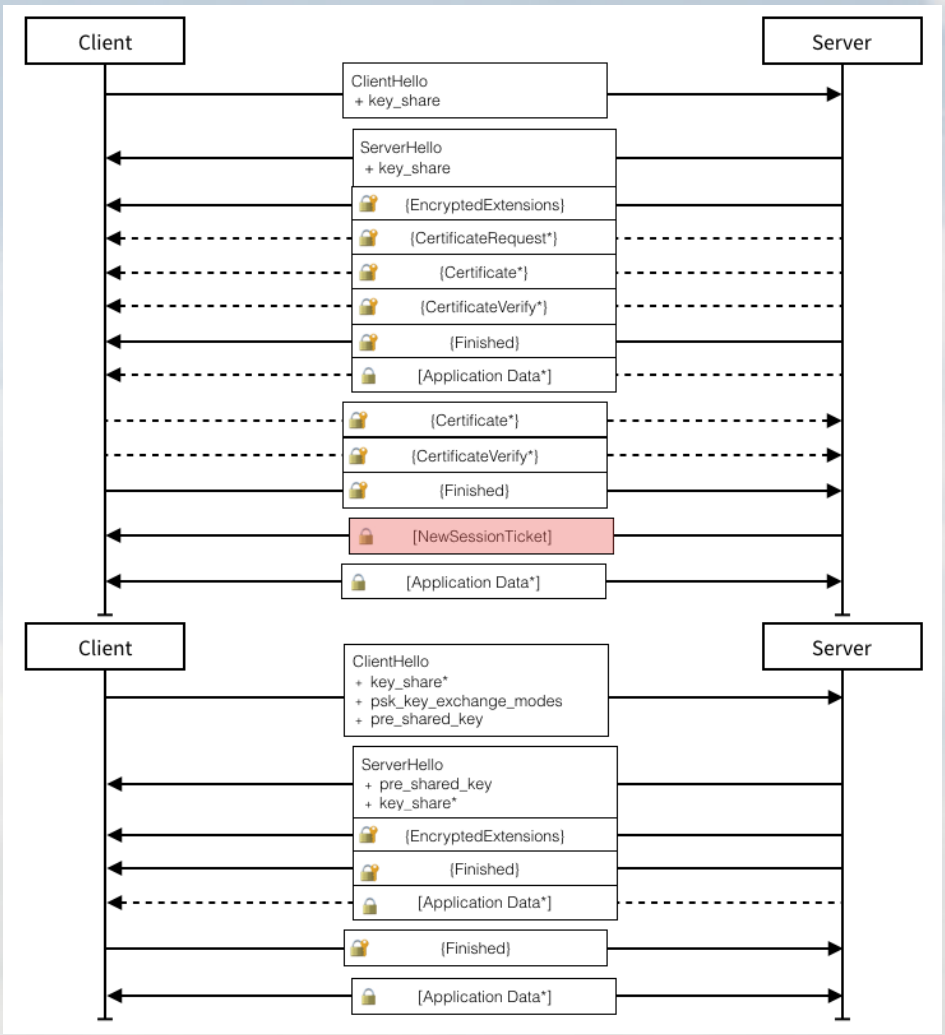
fuente: https://www.davidwong.fr/tls13/#section-2

Full Handshake rechazado :



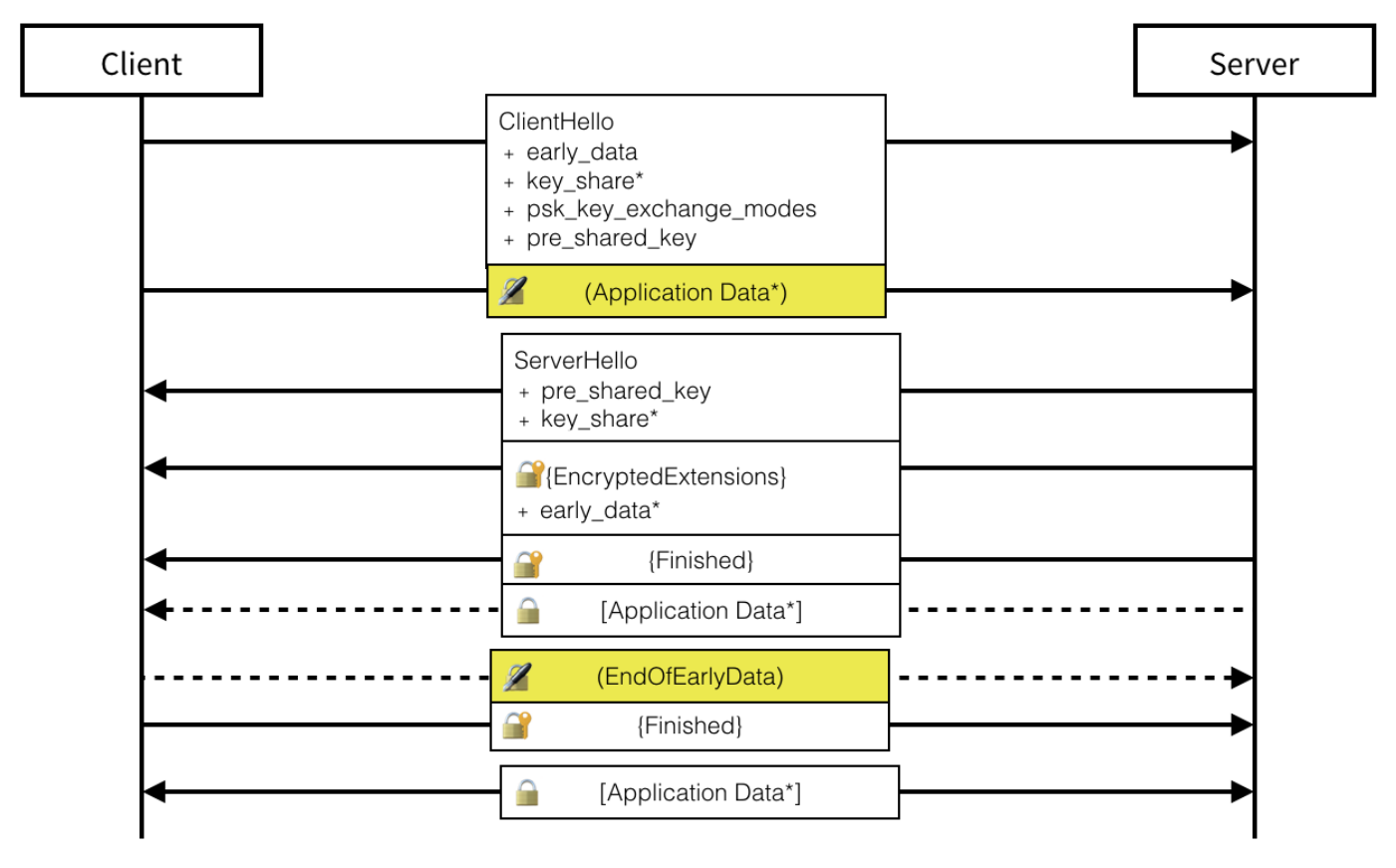
fuente: https://www.davidwong.fr/tls13/#section-2

Resumption and PSK:



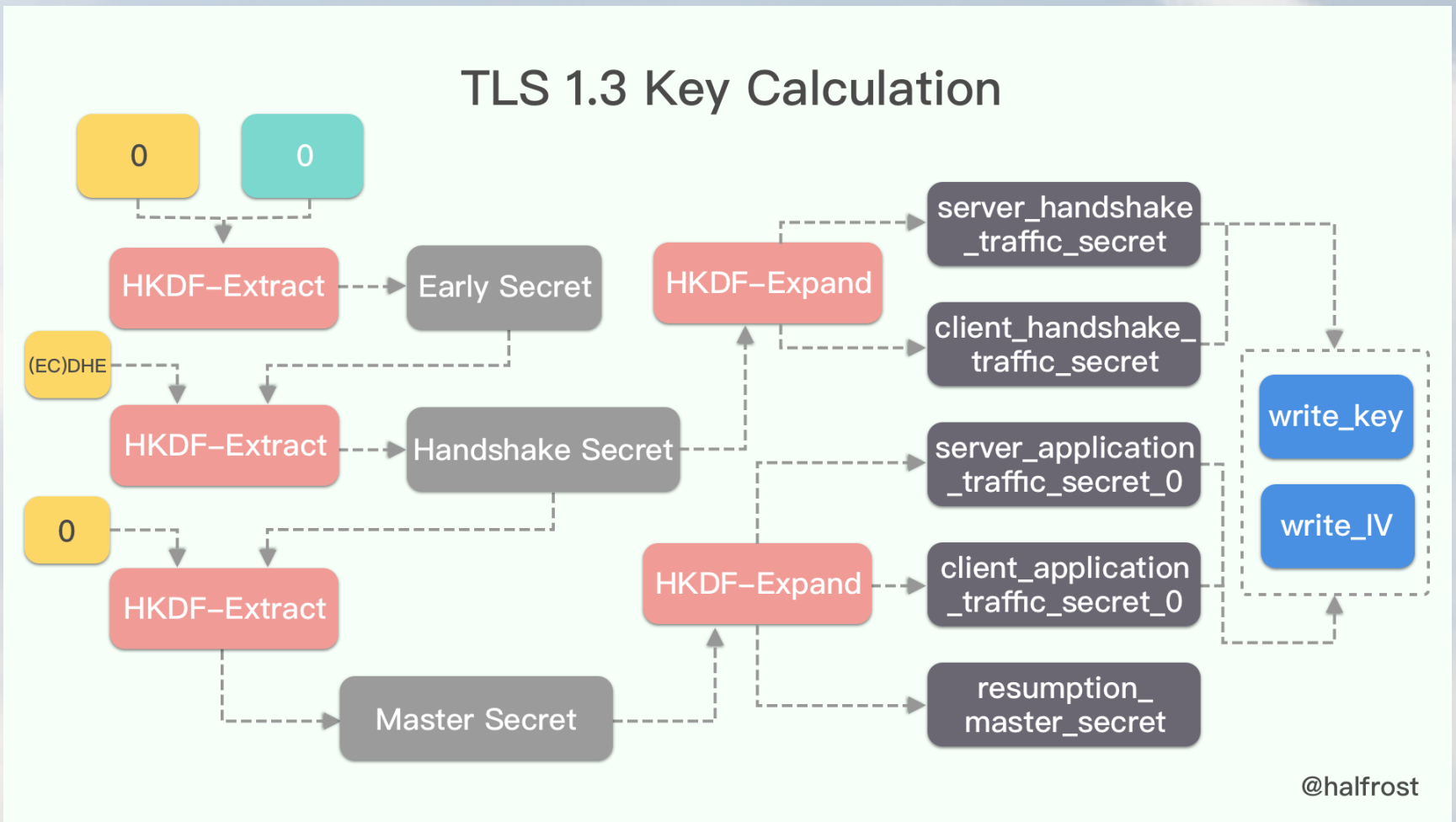
fuente: https://www.davidwong.fr/tls13/#section-2

0-RTT Data



fuente: https://www.davidwong.fr/tls13/#section-2

Generación parámetros criptográficos



source: https://github.com/halfrost/Halfrost-Field/blob/master/contents/Protocol/HTTPS-key-cipher.md

Seguridad

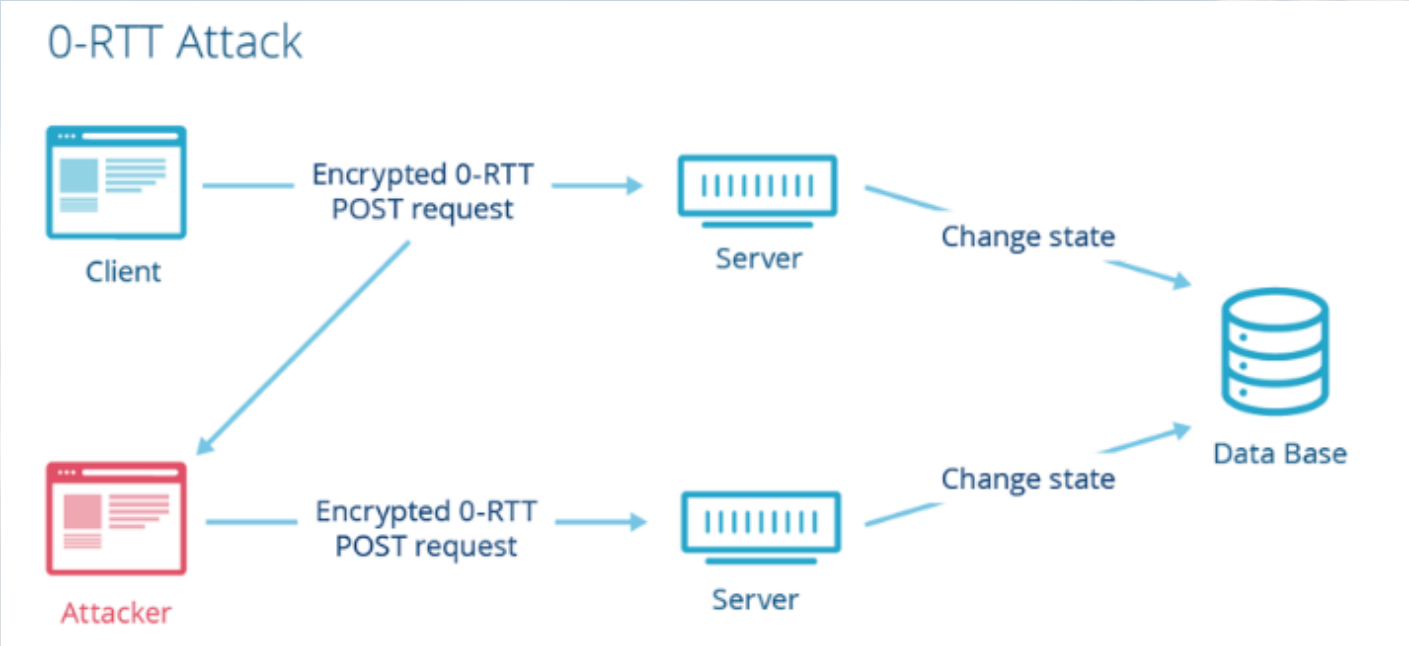
Vulnerabilidades y ataques sobre TLSv1.3:

- *0-RTT reply attack*
- *In March/April 2019, Nir Drucker and Shay Gueron released a paper in which they identify a security vulnerability in the PSK feature of TLS 1.3 that can be exploited in a new reflection attack called Selfie Attack.*
- Ataques a claves RSA (no permitidas en v1.3), una variación del ataque *Bleichenbacher Oracle Attack* en base a *downgrade-attack to v1.2*
- Amenaza "Harvest Now, Decrypt Later" (HNDL). Solución: *Hybrid key agreement*

Guidelines for the Selection, Configuration, and Use of TLS implementations(2019)
SSL/TLS ataques criptográficos modernos(2020)



0-RTT Reply attack

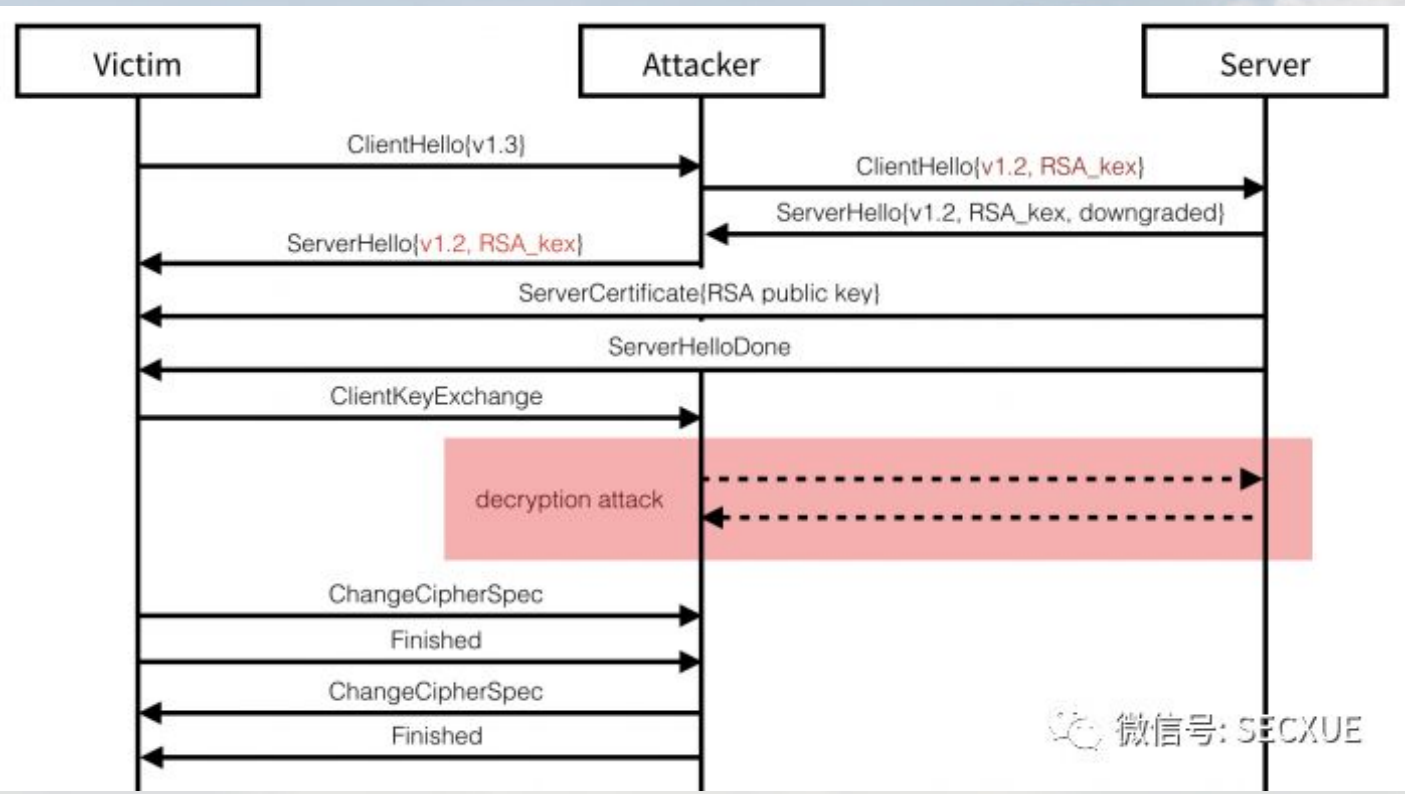


fuente: *https://halfrost.com/tls_1-3_0-rtt/*

Anti-replay measures

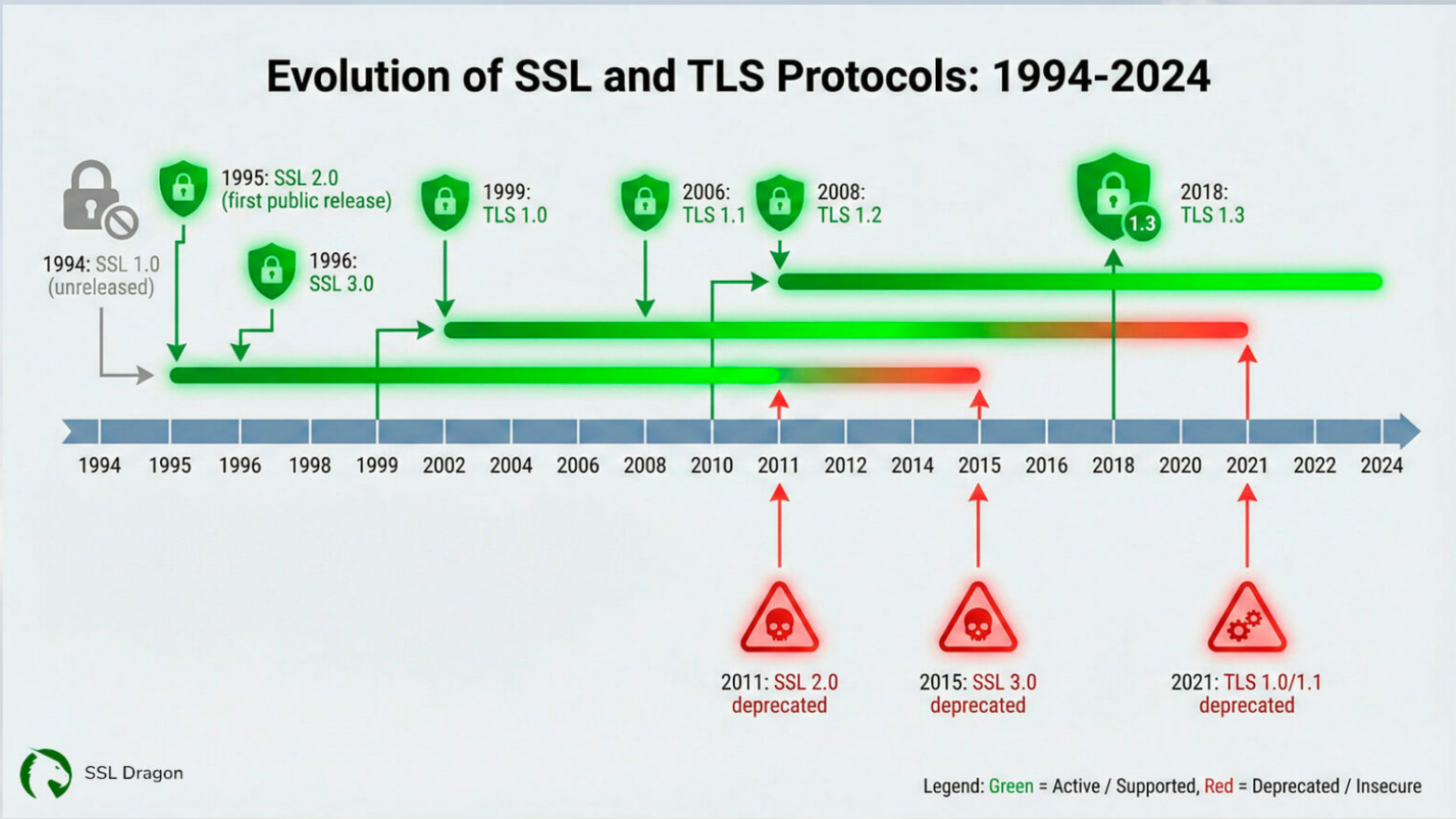


Downgrade attack



fuente: *vulnerabilities in tls libraries and downgrade attack*



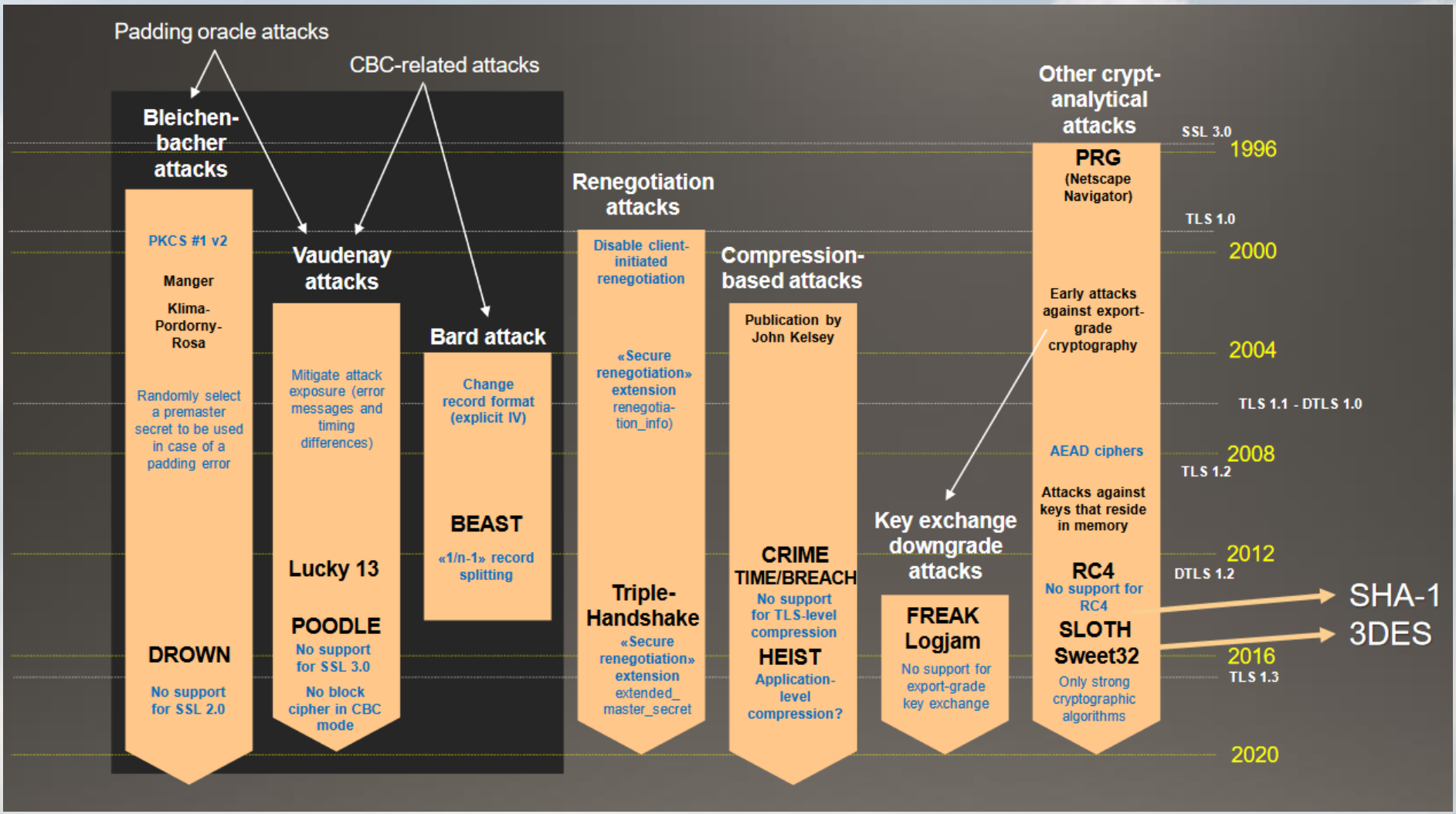


https://www.ssldragon.com/es/blog/historial-de-versiones-ssl-tls/

SSL, TLS and PKI history

Recommendations for Secure Use of TLS and DTLS (RFC 9325)



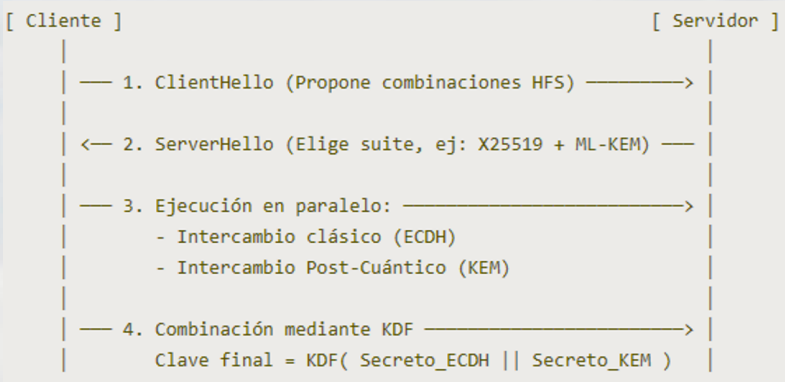


fuente: https://www.esecurity.ch/TLS1.3.pdf

Post-Quantum Cryptography in TLSv1.3

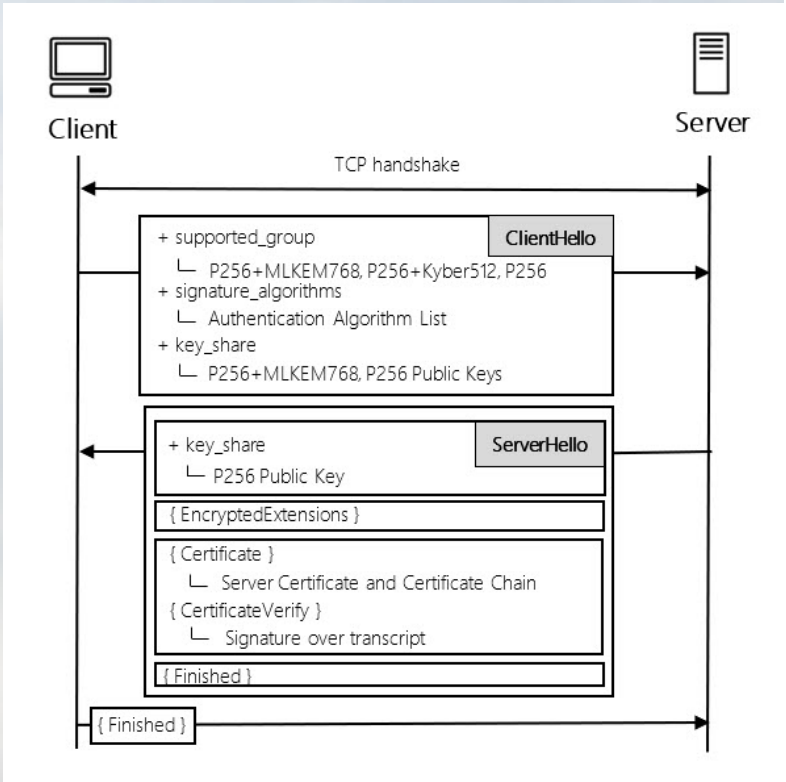
Handshake protocol: *Se aplica únicamente durante esta fase (ML-KEM para el intercambio de claves o ML-DSA para firmas/certificados)*
Record protocol*: se mantiene el uso de criptografía simétrica, intrínsecamente segura contra ataques cuánticos*

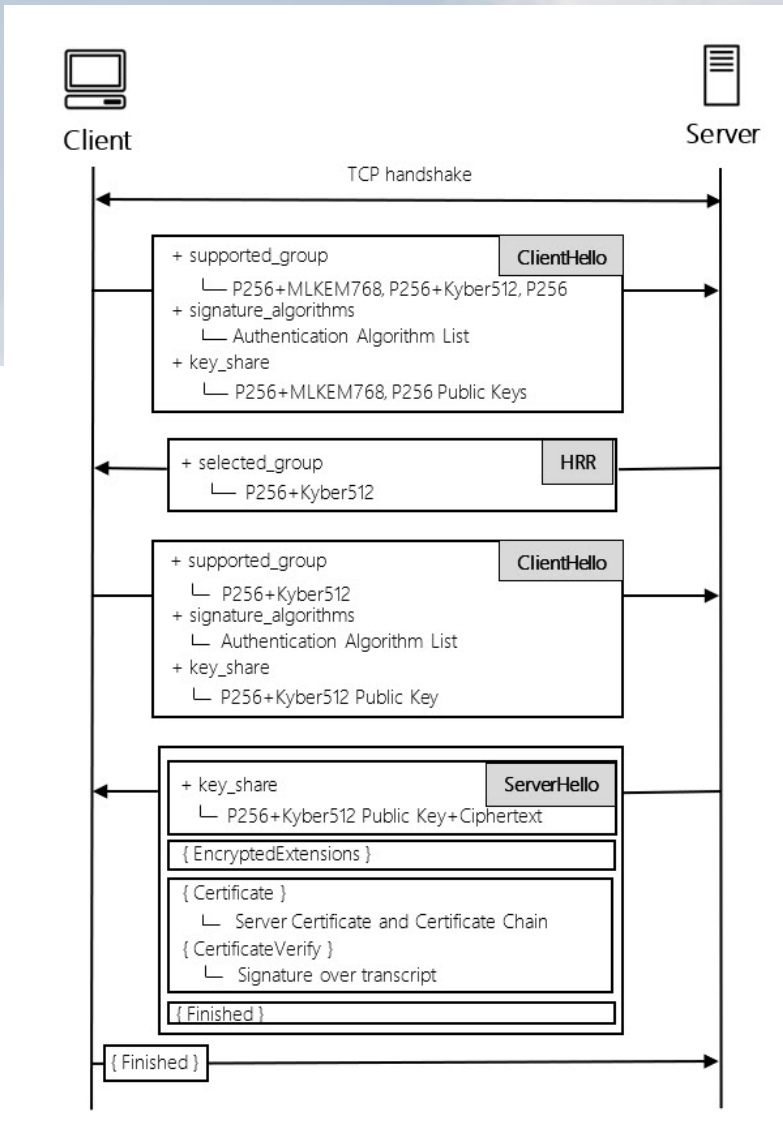
Hybrid key agreement (garantiza *HFS: Hybrid forward secrecy)*



(RFC 9958) Post-Quantum Cryptography for Engineers (Jn 26)
(RFC 9954) Hybrid Key Exchange in TLS 1.3 (Jl 26)
(RFC 10024) PQ/T hybrid key agreement for TLS 1.3 (Ag26)
(RFC draft) ML-KEM Post-Quantum Key Agreement for TLS 1.3
(RFC draft) ***Use of ML-DSA in TLS 1.3 (para autenticación)***

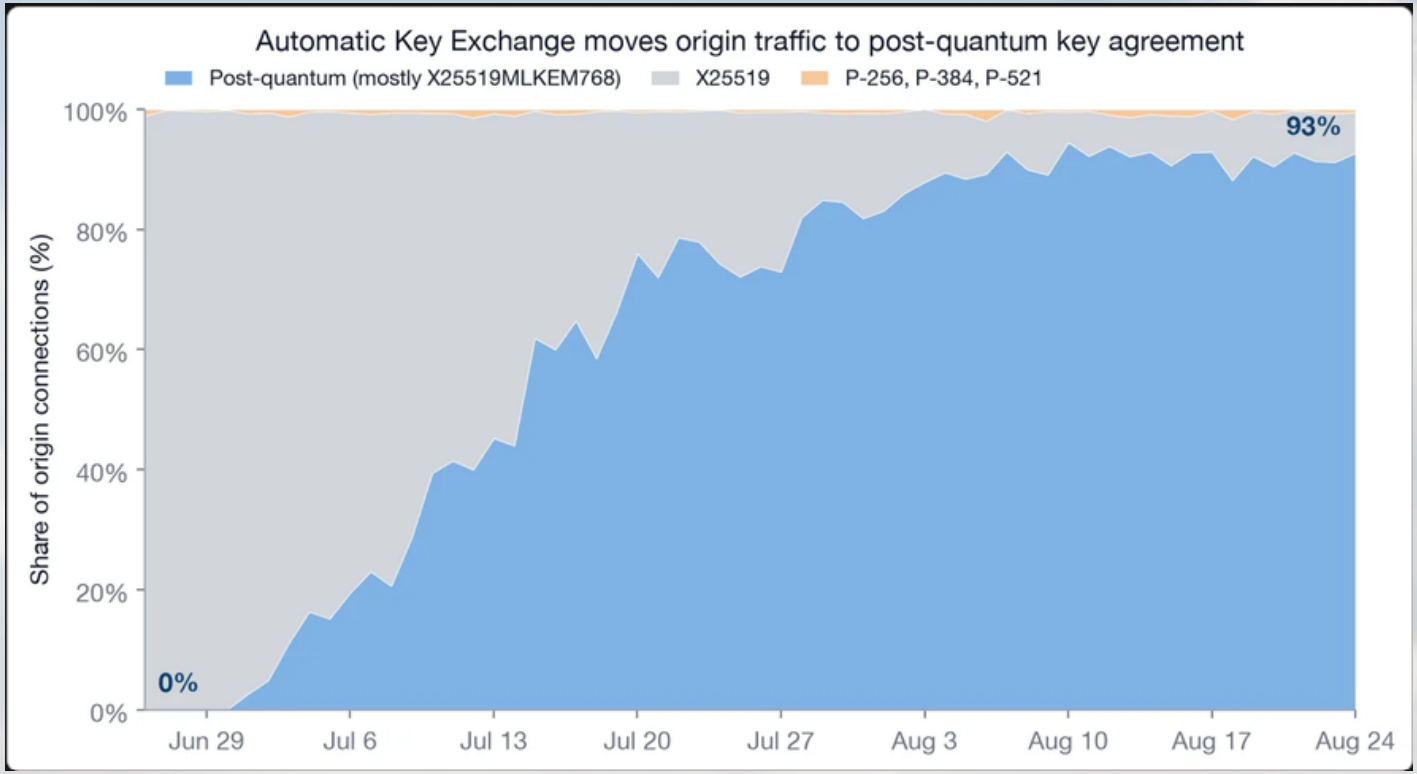






fuente: AWS-paper

En 2026



fuente: https://blog.cloudflare.com/automatic-key-exchange-for-origins/

Firefox: *Web Developer Tools* → *Network* → *Security*

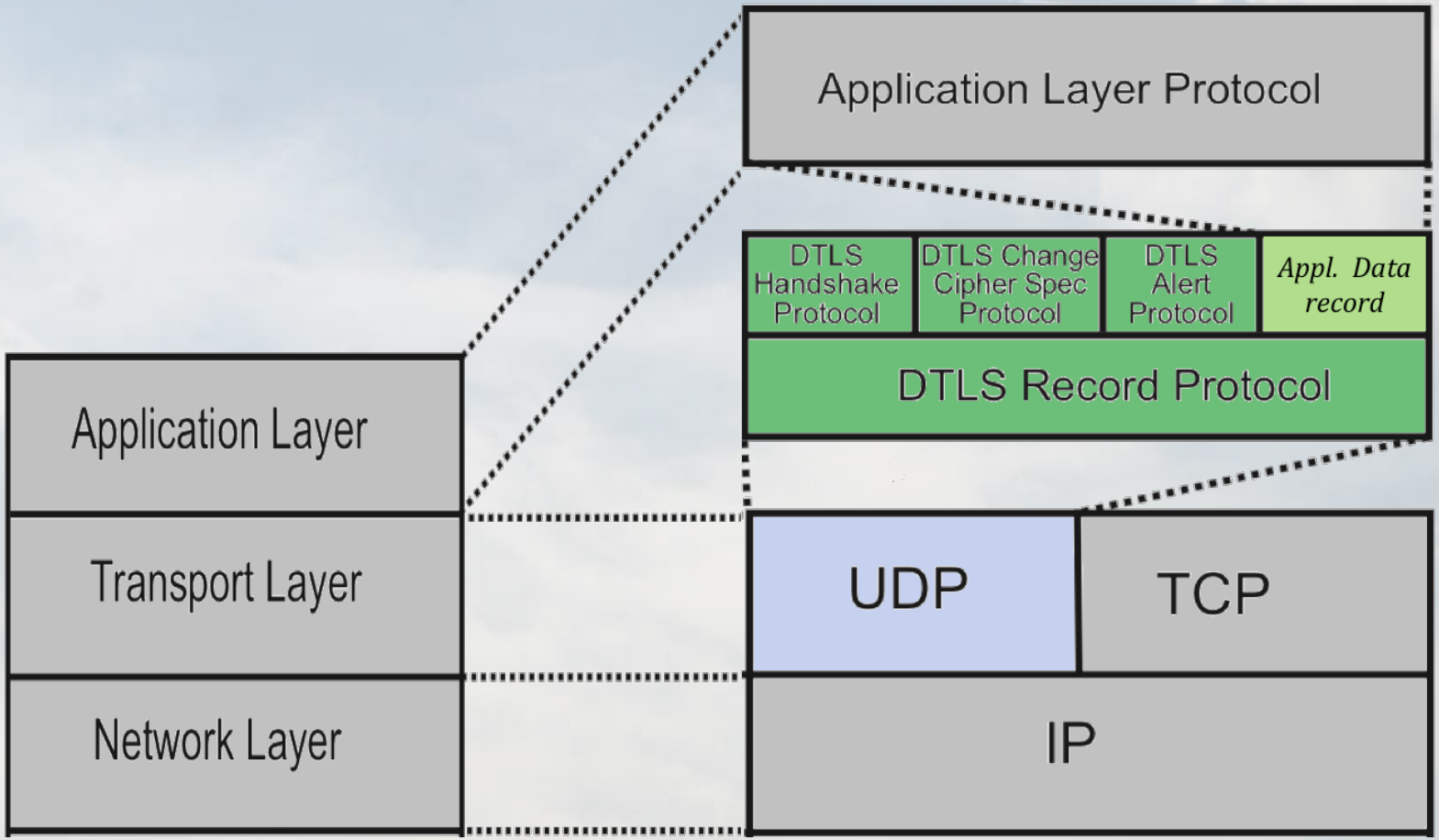


Datagram TLS (v 1.2 - RFC 6347-Obsolete)

- Basado en TLSv1.2
- Ofrece los mismos servicios de **seguridad** (integridad, autenticación y confidencialidad) a las aplicaciones, pero bajo el protocolo UDP
- DTLS proporciona la **fiabilidad** que no aporta UDP:
  - Pérdida datagramas: Temporizador + Retransmisión
  - Desorden por fragmentación, *replay attack*: [#epoch + #secuencia] + Reordenar
  - DoS: *cookie*
- *DTLS handshake messages are grouped into a series of message flights. A flight starts with the handshake message transmission of one peer and ends with the expected response from the other peer or ACK*
- Orientado hacia aplicaciones sensibles a retardos que trabajan sobre protocolos de transporte no fiables: video-conferencia, telefonía vía Internet, juegos online...
  - Alternativas: usar IPsec, diseñar un protocolo de aplicación seguro personalizado, o rehacer la aplicación sobre TCP para usar TLS



Arquitectura DTLS

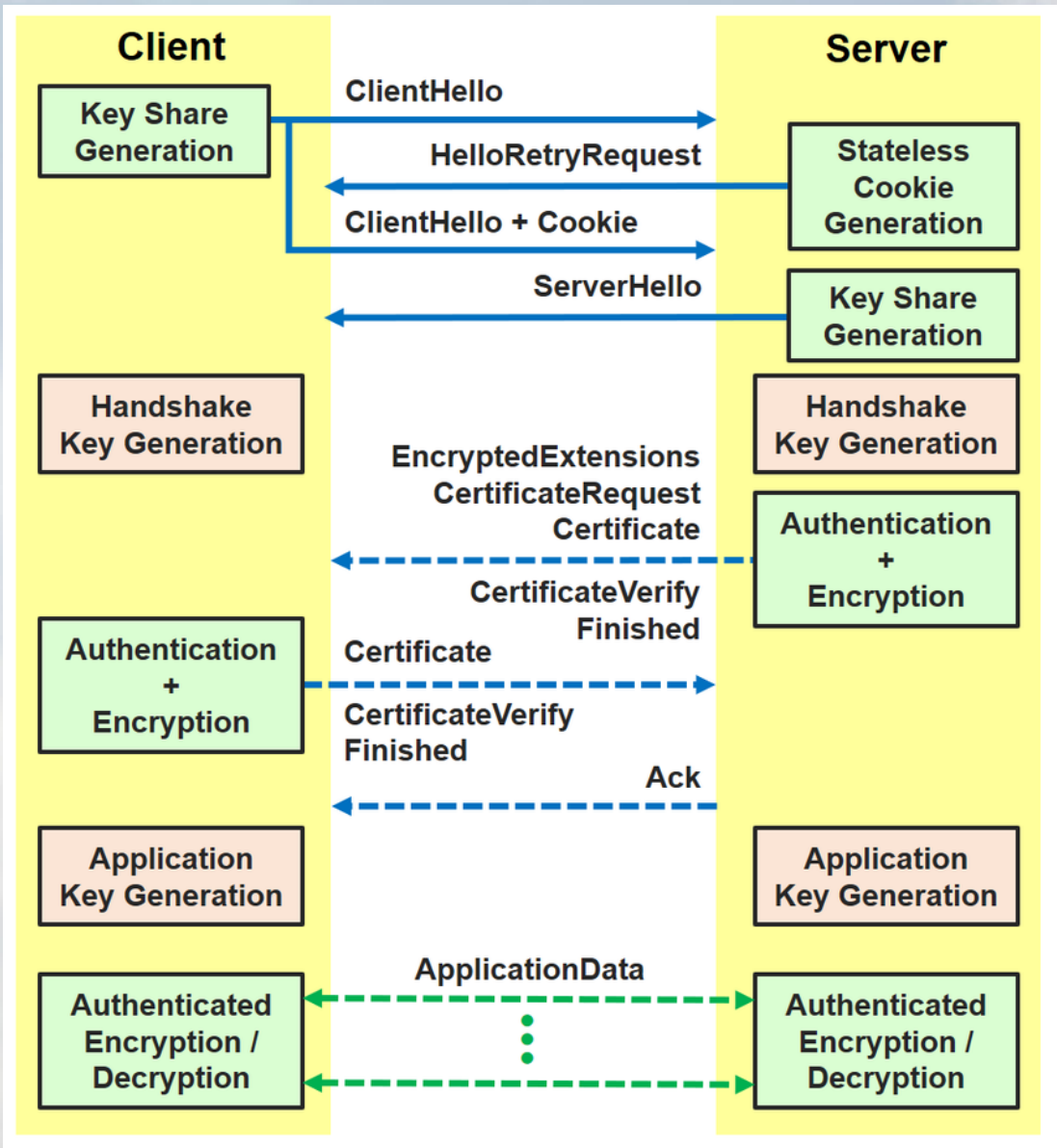


fuente: "SSL and TLS: Theory and Practice"- Oppliger 2009

DatagramTLS (v 1.3 - *RFC 9147*)

- Basado en TLS 1.3 (Ab22)
- *Implementations that speak both DTLS 1.2 and DTLS 1.3 can interoperate with those that speak only DTLS 1.2 (using DTLS 1.2).*
- Cambios respecto a DTLSv1.2, siguiendo los cambios de TLS1.2 a TLS1.3
  - *Encrypted sequence number*
  - Solo AEAD para cifrar y autenticar
  - *HelloRetryRequest* sustituye a *HelloVerifyRequest*
  - Función rediseñada para *Key derivation* (HKDF)
  - Mecanismo nuevo de *session resumption* via PSK tickets
  - 0-RTT
  - Descarta el protocolo *ChangeCipherSpec*
  - ***Supported PQC algorithms***
- OpenSSL (v4.1 Oc26), WolfSSL soporta DTLS1.3




Client
Server
Key Share Generation
ClientHello
HelloRetryRequest
Stateless Cookie Generation
ClientHello + Cookie
ServerHello
Key Share Generation
Handshake Key Generation
Handshake Key Generation
EncryptedExtensions
CertificateRequest
Certificate
Authentication + Encryption
CertificateVerify
Finished
Authentication + Encryption
Certificate
CertificateVerify
Finished
Ack
Application Key Generation
Application Key Generation
ApplicationData
Authenticated Encryption / Decryption
Authenticated Encryption / Decryption

fuente

Laboratorio

Implementar en C un protocolo de aplicación mediante el uso de la API de SSL/TLS (openssl)



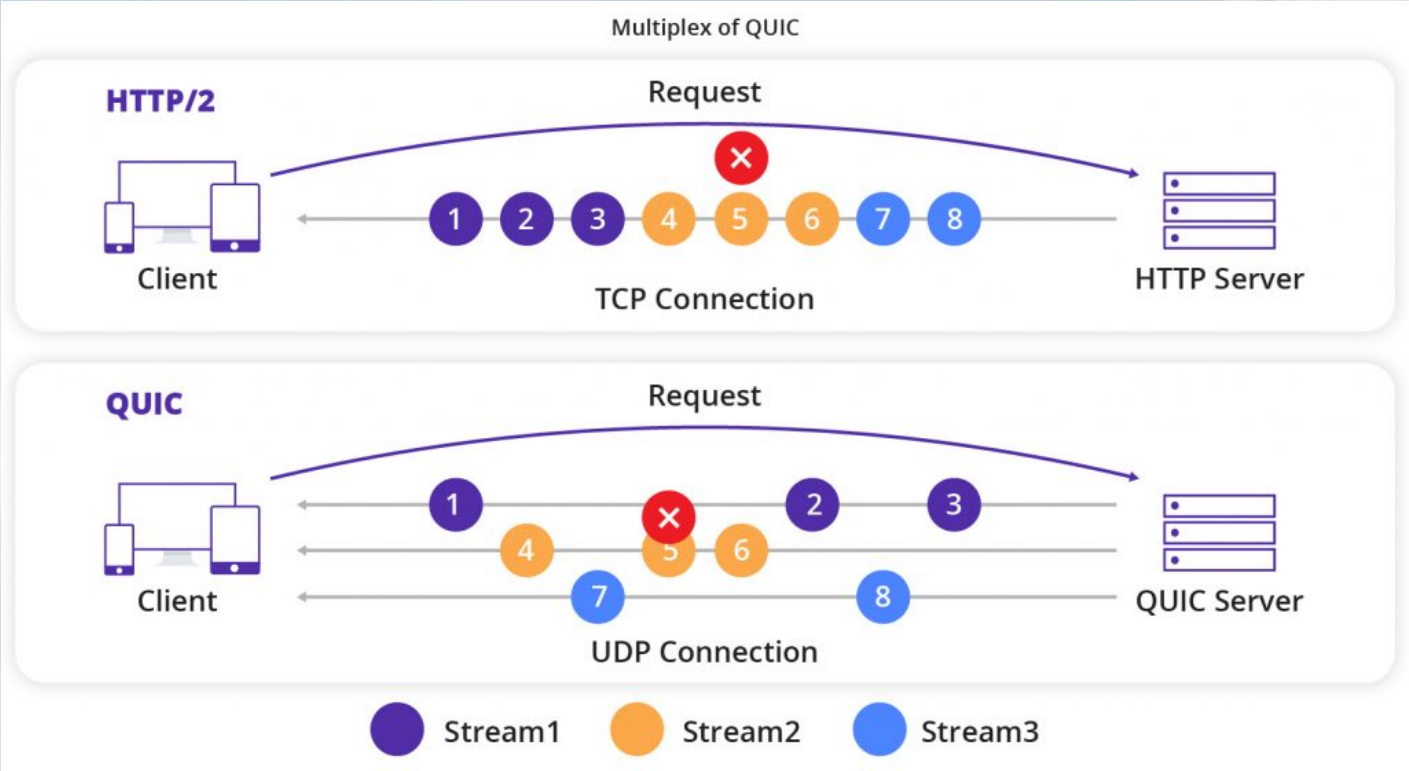
QUIC(RFC 9000-2, 9369)

QUIC protocolo de transporte fiable y seguro (equivalente a TLS).
Características:

- En un protocolo orientado a conexión que establece una sesión entre cliente y servidor
- Establece conexiones fiables sobre UDP:
  - Identificador conexión 64bits
  - Multiplexa diferentes tipos de ***streams*** (flujos lógicos) para transportar datos en una única conexión física
    - ★ Control independiente de flujos: cada flujo se entrega a la aplicación de forma fiable y en orden. No se controla el orden entre flujos.
- Fiabilidad: mejoras en la detección de pérdida de paquetes y control de congestión (RFC 9002)

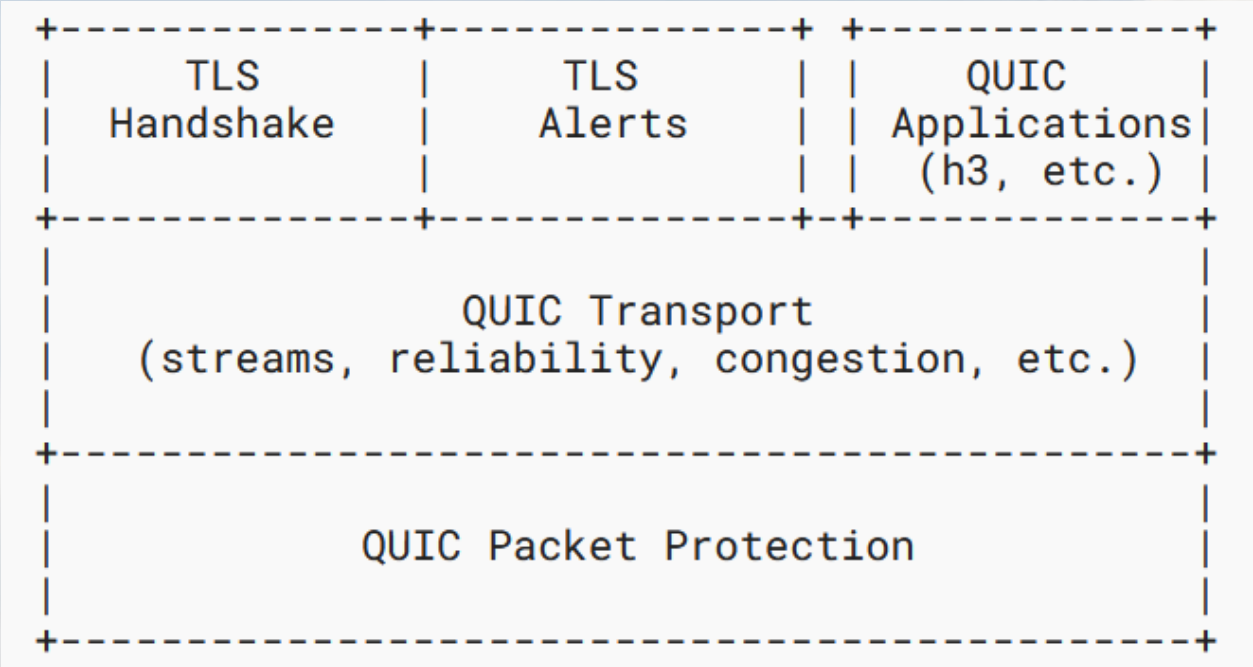


Multiplexación: HTTP/2 vs QUIC



fuente: https://www.cdnetworks.com/blog/media-delivery/what-is-quic/

- *QUIC layers*:



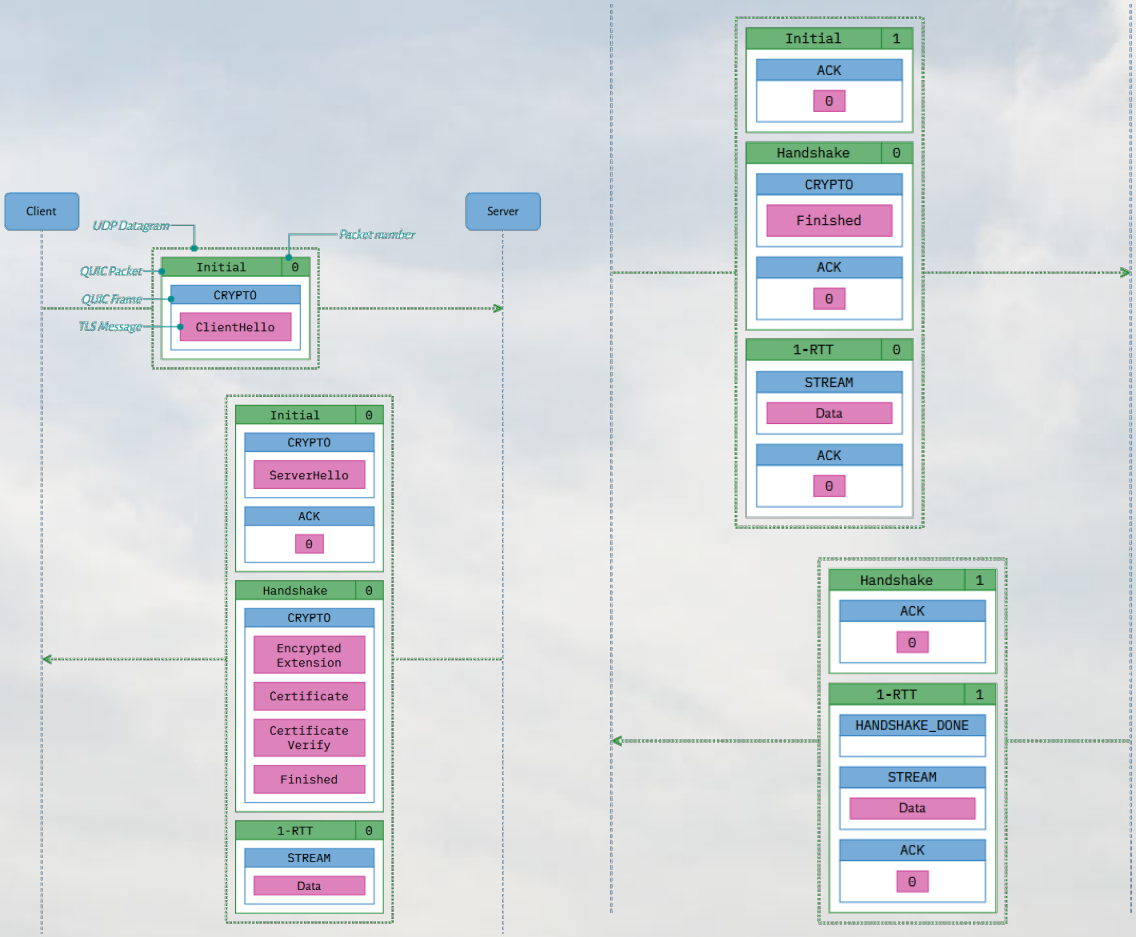
fuente: RFC 9001

- Reduce la latencia en el establecimiento de la conexión y en el transporte de datos (1-RTT y 0-RTT)
- *Handshake*: combina la negociación de parámetros criptográficos y de transporte
  - Integra el handshake de **TLS1.3** (RFC 9001 - Using TLS to secure QUIC)
  - Estos mensajes se encapsulan en *CRYPTO frames*
- No aplica "*TLS Record protocol*", define su propio formato de tramas especializadas para cada función
  - Mensajes de aplicación se encapsulan en *STREAM frames*
  - Además del *payload*, también están autenticadas y cifradas las cabeceras de las tramas



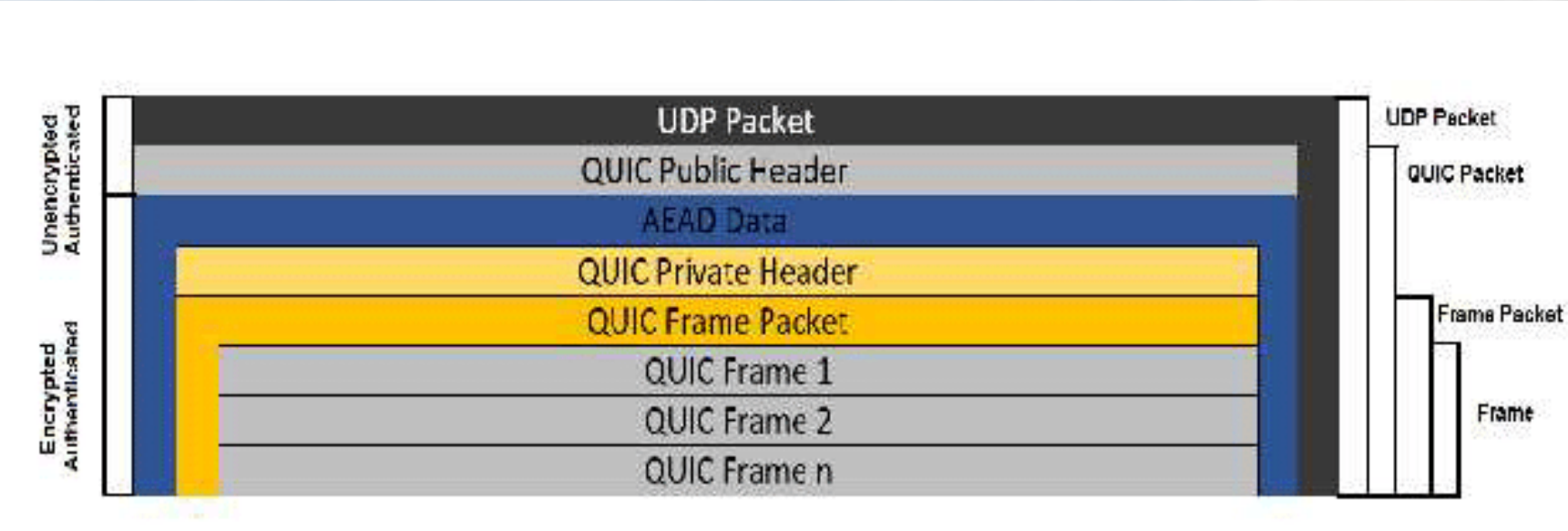
en detalle

- QUIC *Handshake*



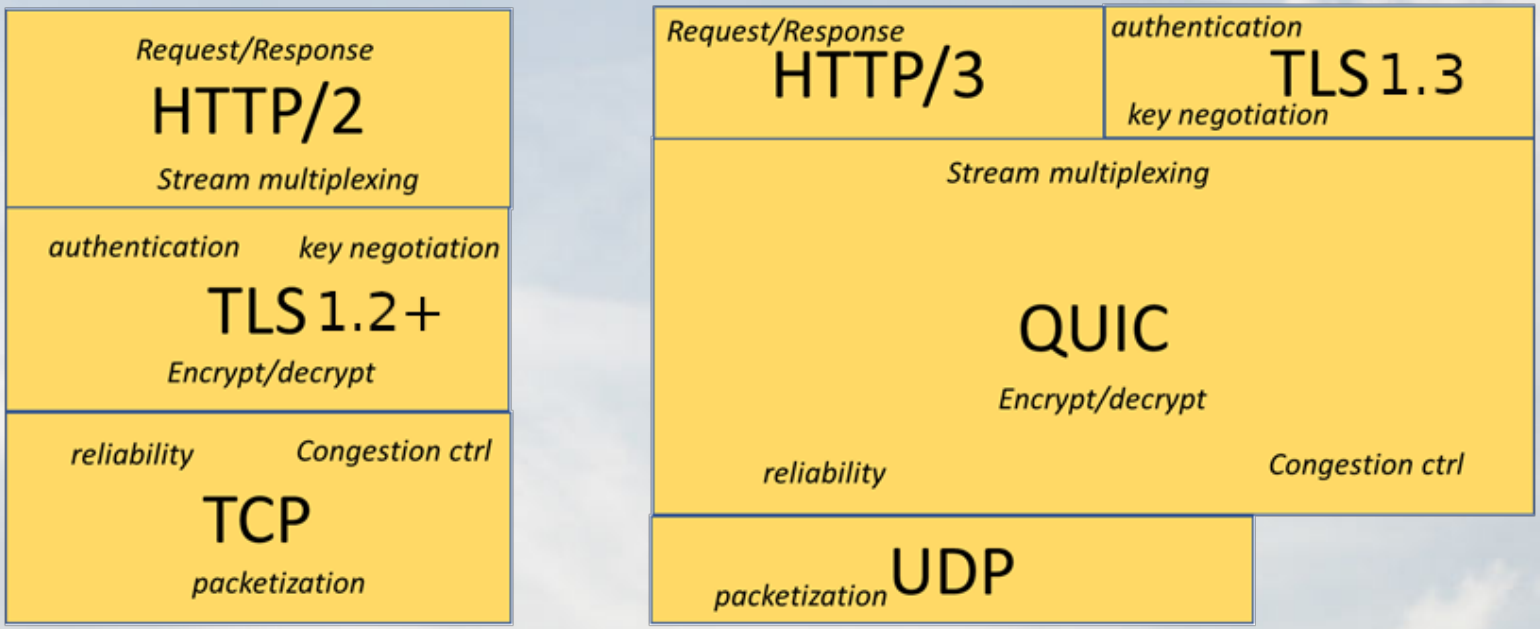
fuente: http3-in-practice-quic

- QUIC *packet format*:



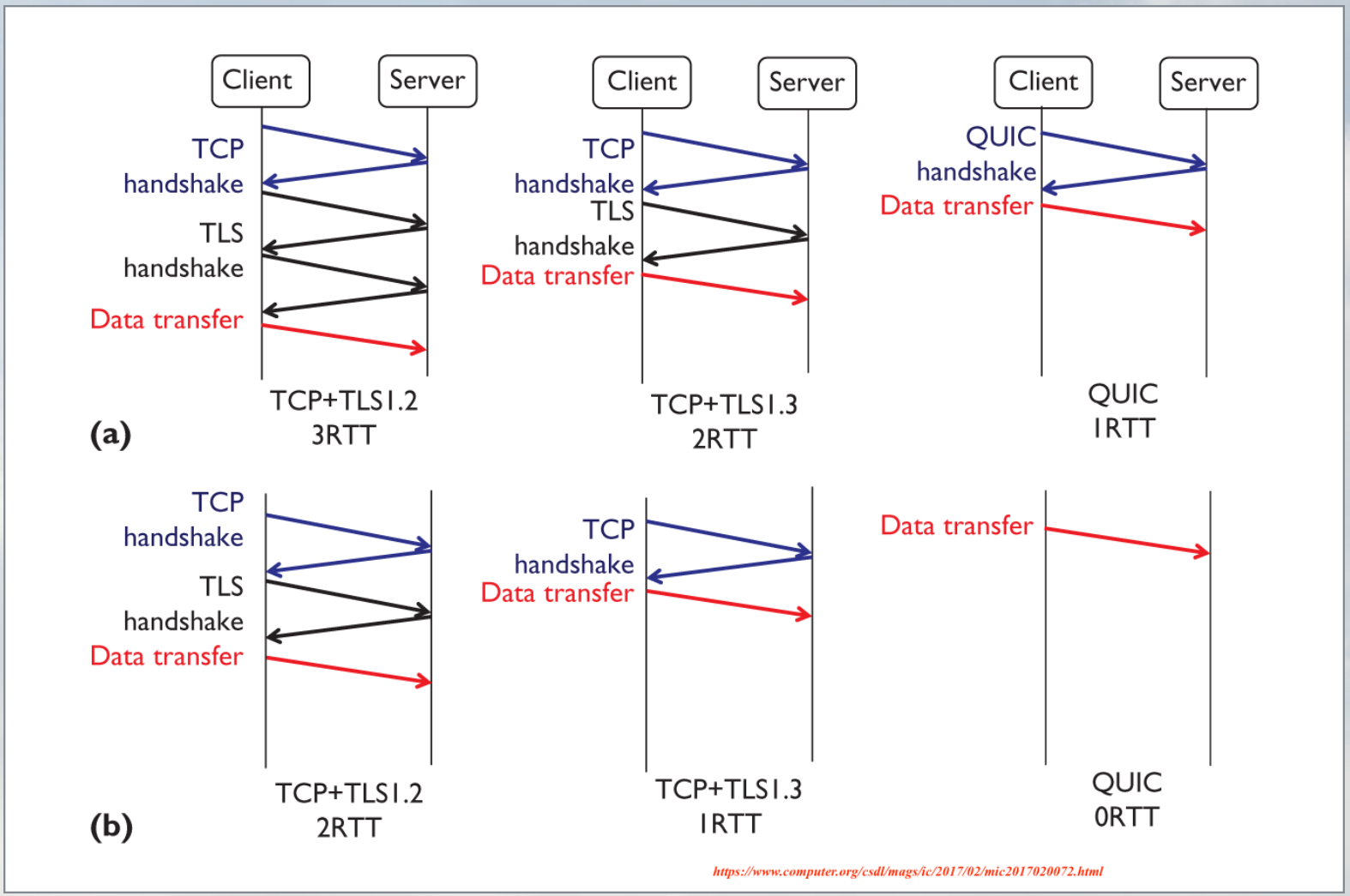
fuente: *Performance analysis of QUIC*

HTTP-over-QUIC (HTTP/3 explained)



fuente: https://www.f5.com/es_es/company/blog/quic-will-eat-the-internet

Detalle comparativa



Adopción y uso de protocolos

QUIC y PQC

- En el *handshake* de TLS1.3 utiliza *hybrid key share* (p.e.: X25519MLKEM768)
  - Concatenación de claves: aumenta el tamaño de *ClientHello* (de 1200B a ~2KB), lo que implica fragmentarlo en varios paquetes QUIC, que el servidor deberá reensamblar.
    Algunos dispositivos de seguridad descartan mensajes *ClientHello* que exceden un tamaño dado o no admiten *multi-packet ClientHellos*
  - *ServerHello* aumenta exponencialmente su tamaño: intercambio de claves (~1KB) + certificado (firma ML-DSA-65: 2.4KB~3.3KB) * cadena de certificados
- [Oc26, *PQC signature* aún no implementada para autenticación]



Metric	Q3 2025	Q2 2026	Q3 2026	QoQ	YoY
HTTP/2	50.47%	51.85%	**52.46%**	+0.61 pt	+1.99 pt
HTTP/1.x	26.55%	26.24%	**27.37%**	**+1.13 pt**	+0.83 pt
HTTP/3	22.98%	21.91%	**20.17%**	**−1.74 pt**	**−2.81 pt**
HTTPS	93.97%	94.89%	**95.10%**	+0.21 pt	+1.13 pt
TLS 1.3	68.12%	71.29%	**72.70%**	+1.40 pt	+4.58 pt
TLS QUIC	24.41%	23.04%	**21.19%**	−1.85 pt	−3.22 pt
TLS 1.2	7.44%	5.65%	**6.10%**	+0.45 pt	−1.34 pt
Post-quantum key exchange (client leg)	32.04%	55.34%	**57.30%**	+1.96 pt	+25.26 pt
Post-quantum at origins (X25519MLKEM768)	4.87%	9.58%	**11.91%**	+2.32 pt	+7.04 pt
Bot share of requests	30.13%	33.21%	**35.49%**	+2.28 pt	+5.36 pt

fuente: Protocol adoption



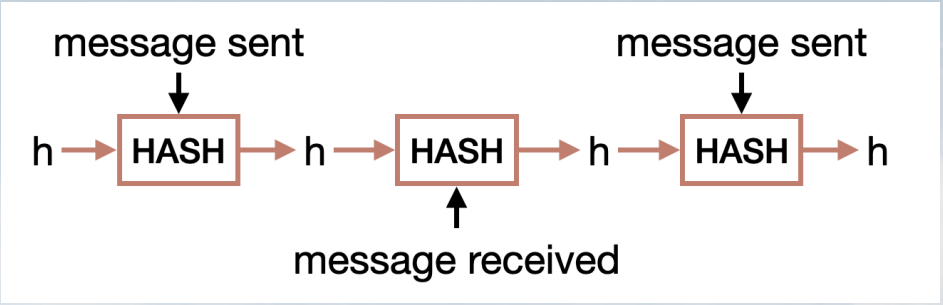
Noise protocol framework

- Alternativa a TLS
- No es un protocolo, es un *framework* para construir *crypto-protocols:*
  - Ofrece diferentes patrones de *handshake*
    En la fase de diseño, el desarrollador determina cuál utilizar en su aplicación.
  - Durante su uso, no hay negociaciones en el *handshake*, se sigue el protocolo diseñado.
- Utilizado por WhatsApp, WireGuard, I2P y Lightning

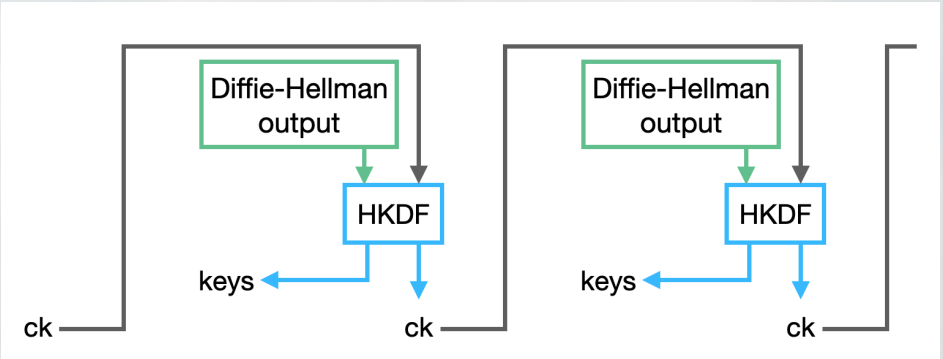
Noise protocol Framework



- Autenticación continuada de los mensajes del *handshake*, en ambos extremos mantienen h (*hash*) que es utilizada como *associated data* en un AEAD



- En cada *DH key exchange* se deriva un nuevo conjunto de claves simétricas para autenticar y cifrar los siguientes mensajes en base a ck (*chaining key*).



fuente: real-world-cryptography chapter-9

- Propuestas para incorporar mecaniscos cuánticos: NoisePQC++, libreria experimental